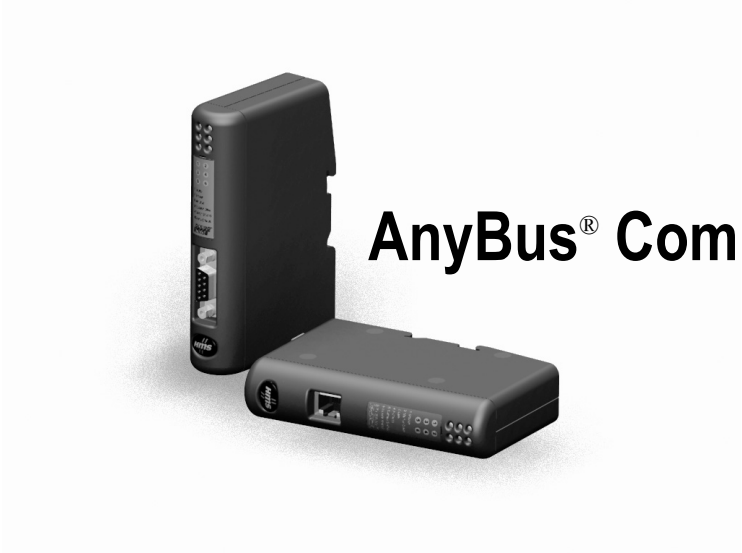


User Manual
AnyBus® Communicator for Profibus

Rev. 2.02

HMS Industrial Networks AB

	☎	✉
Germany	+49- 721 - 96472 - 0	sales-ge@hms-networks.com
Japan	+81- 45 - 478 -5340	sales-jp@hms-networks.com
Sweden	+46- 35 - 17 29 20	sales@hms-networks.com
U.S.A	+1- 773 - 404 - 3486	sales-us@hms-networks.com

Table of Contents

Chapter 10 Advanced Functions

Appendix A Configuration Wizards

Appendix B Troubleshooting

Appendix C Connector Pin Assignments

Appendix D Technical Specification

Appendix E ASCII Table

About This Manual

How To Use This Document

This document contains a general introduction as well as a description of the technical features provided by the AnyBus Communicator module, including the PC-based configuration software.

This document shall be used together with the appendix of the respective fieldbus type.

The reader of this document is expected to be familiar with PLC and software design, as well communication systems in general. The reader is also expected to be familiar with the Microsoft Windows operating system.

Important User Information

The data and illustrations found in this document are not binding. We, HMS Industrial Networks AB, reserve the right to modify our products in line with our policy of continuous product development. The information in this document is subject to change without notice and should not be considered as a commitment by HMS Industrial Networks AB. HMS Industrial Networks AB assumes no responsibility for any errors that may appear in this document.

There are many applications of this product. Those responsible for the use of this device must ensure that all the necessary steps have been taken to verify that the application meets all performance and safety requirements including any applicable laws, regulations, codes, and standards.

AnyBus® is a registered trademark of HMS Industrial Networks AB. All other trademarks are the property of their respective holders.

Revision Notes

Revision	Date	Author	Chapter	Description
2.00	2003-06-24	PeP	All	Second major release
2.01	2004-03-08	PeP	20	Corrected Control Codes
2.02	2005-02-04	PaL	Appendix C	Added recomended Profi bus connectors

Related Documentation

Document name	Author
ABC-PDP Installation Leaflet	HMS

Support

If technical support is required, see the web FAQ (www.hms-networks.com), or please contact the nearest Support Centre:

Europe (Sweden)

E-mail: support@hms-networks.com
Phone: +46 (0) 35 - 17 29 20
Fax: +46 (0)35-17 29 09
Online: www.hms-networks.com

HMS America

E-mail: us-support@hms-networks.com
Phone: +1-773-404-2271
Toll Free: 888-8-AnyBus
Fax: +1.773.404.1797
Online: www.hms-networks.com

HMS Germany

E-mail: ge-support@hms-networks.com
Phone: +49-721-96472-0
Fax: +49 721 964 7210
Online: www.hms-networks.com

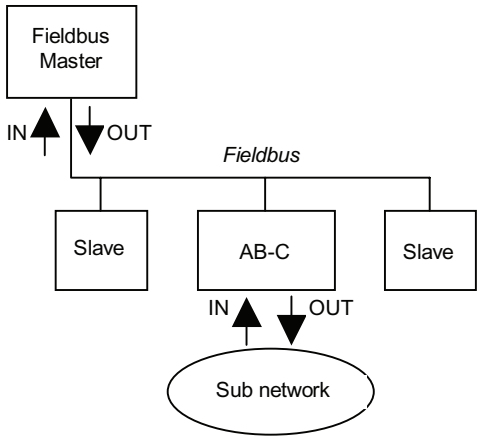
HMS Japan

E-mail: jp-support@hms-networks.com
Phone: +81-45-478-5340
Fax: +81 45 476 0315
Online: www.hms-networks.com

Conventions Used in This Document

The following conventions are used throughout this document:

- Numbered lists provide sequential steps
- Bulleted lists provide information, not procedural steps
- The term 'user' refers to the person or persons responisble for installing the AnyBus Communicator in a network.
- Hexadecimal values are written in the format 0xNNNN, where NNNN is the hexadecimal value.
- Decimal values are represented as NNNN where NNNN is the decimal value
- As in all communication systems, the terms "input" and "output" can be ambiguous, because their meaning depend on which end of the link is being referenced.
  The convention in this document is that "input" and "output" are always being referenced to the master/scanner end of the link. *(see illustration below)*



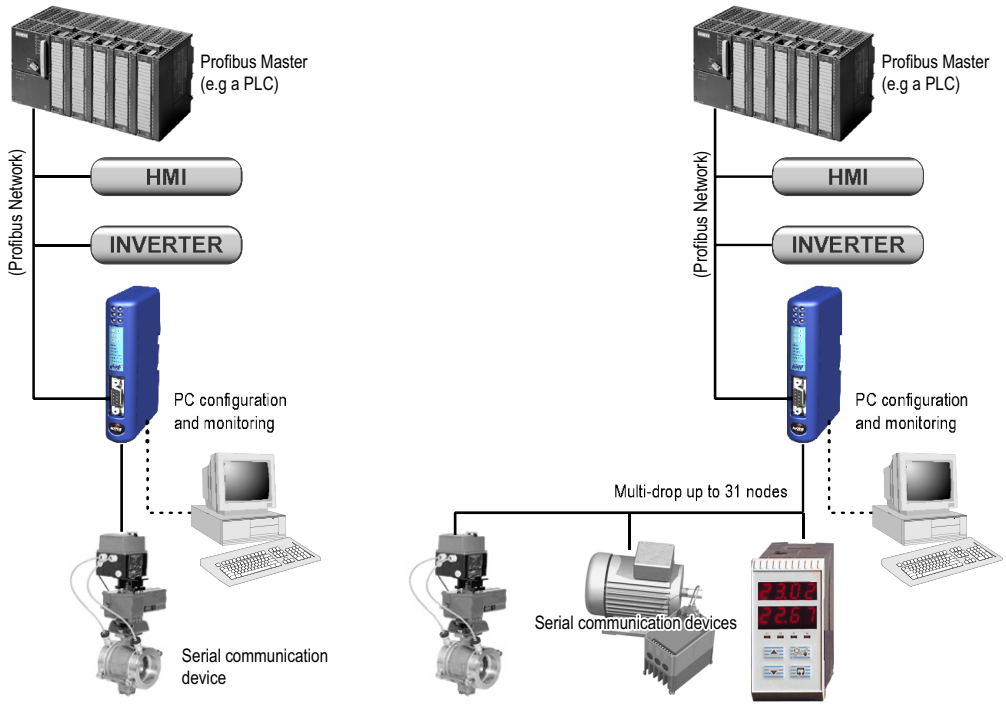
Input and Output definition

Glossary

Term	Meaning
ABC	AnyBus Communicator Module
Broadcaster	A protocol specific node in the sub-network scan- that hold transactions destined to all nodes.
Command	A protocol specific Transaction.
Fieldbus	The network to which the communicator is connected.
Frame	Higher level series of bytes forming a complete telegram on the sub-network
Mailbox	A HMS specific entity that is used for communication and configuration of the AnyBus-S module.
Monitor	A tool for debugging the ABC and the network connections.
Node	A device in the scan-list that defines the communication with a slave on the sub-network
Scan list	List of configured Slaves with transactions on the sub-network.
Sub-network	The network that logically is located on a subsidiary level with respect to the fieldbus and to which the ABC acts as a gateway.
Transaction	A generic building block that is used in the sub-network scan-list and defines the data that is sent out the sub-network.
Fieldbus Control System	Fieldbus master

About the AnyBus Communicator for Profibus

The AnyBus Communicator for Profibus or ABC acts as a gateway between almost any serial application protocol and a Profibus-DP network. Integration of industrial devices is enabled without loss of functionality, control and reliability, both when retro-fitting to existing equipment as well as when setting up new installations.



Single-Node Serial Sub Network **Multi-Node Serial Sub Network**

General Features

- DIN-rail mountable
- Fully interchangeability with AnyBus Communicator modules for other networks
- Save/load configuration in flash
- CU, UL & cUL marked

Sub Network

- RS232/422/485
- Multi-drop (up to 31 nodes) or single-node configurations possible
- Modbus RTU Master mode and Generic Data Mode
- Up to 100 instances (A sub network transaction occupies 1 or 2 instances depending on communication model).
- Configuration via Windows software tool (ABC Config Tool)

Fieldbus Interface Features

- Complete Profibus-DP slave functionality according to IEC 61158
- Node Address range: 1-99 using on board switches
- Baudrate range: 9.6 kbit-12Mbit. Auto baudrate detection supported.
- Transmission media: Profibus bus line, type A or B specified in IEC 61158

Connectors

For wiring and pin assignments, see Appendix C-1 “Connector Pin Assignments”.

A: Profibus Connector

This connector is used to connect the ABC to the fieldbus.

B: PC Connector

This connector is used to connect the ABC to a PC for configuration and monitoring purposes.

C: Subnet Connector

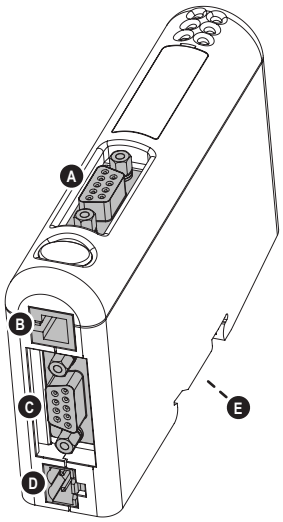
This connector is used to connect the ABC to the serial sub network. (See 4-1 “Sub-network Configuration”)

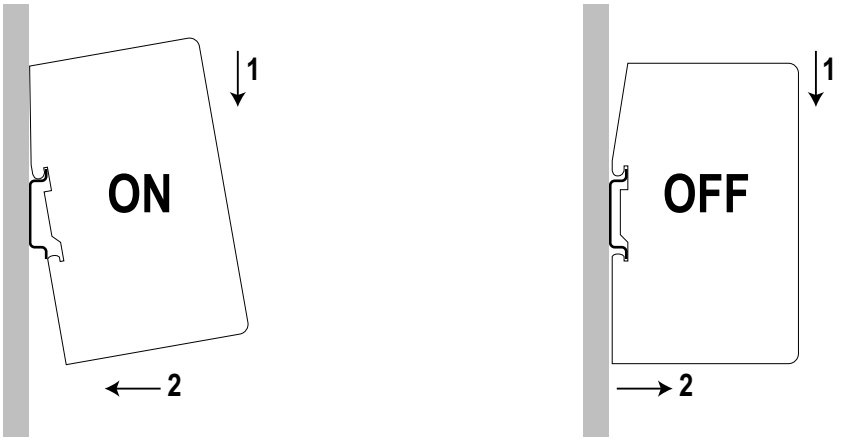
D: Power Connector

Use this connector to apply power to the ABC. (See Appendix D-1 “Technical Specification”.

E: DIN Rail Connection

The din rail connector is internally connected to PE.



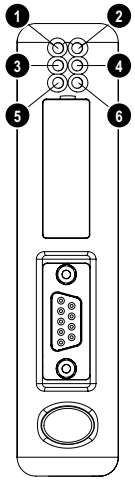


- To snap the ABC *on*, first press the ABC downwards (1) to compress the spring on the DIN-rail connector, then push the ABC against the DIN-rail as to make it snap on (2)
- To snap the ABC *off*, push the ABC downwards (1) and pull it out from the DIN-rail (2), as to make it snap off from the DIN-rail.

Status Indicators

#	State	Status
1 - Fieldbus Online	Off	Not online
	Green	Online
2 - Fieldbus Offline	Off	Not offline
	Red	Offline
3 - (Not used)	-	-
4 - Fieldbus Diag	Off	No diagnostics present
	Red, flashing 1Hz	Error in configuration
	Red, flashing 2Hz	Error in user parameter data
	Red, flashing 4Hz	Error in initialisation
5 - Subnet Status[a]	Off	Power off
	Green, flashing	Initializing and not running
	Green	Running
	Red	Stopped or subnet error, or timeout
6 - Device Status	Off	Power off
	Alternating Red/Green	Invalid or missing configuration
	Green	Initializing
	Green, flashing	Running
	Red, flashing	If the Device status LED is flashing in a sequence starting with one or more red flashes, please note the sequence pattern and contact the HMS support department

a. This led turns green when all transactions have been active at least once. This includes any transactions using “change of state” or “change of state on trigger”. If a timeout occurs on a transaction, this led will turn red.



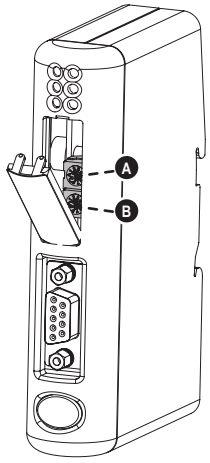
Configuration Switches

The configuration switches are used to set the Profibus node address. Normally, these switches are covered by a plastic hatch. Note that the node address can not be changed during runtime, i.e. the ABC requires a reset for any changes to have effect.

The configuration is done using two rotary switches as follows:

Profibus Node Address = (Switch B x 10) + (Switch A x 1)

Note: When removing the hatch, avoid touching the circuit boards and components. If tools are used when opening the hatch, be cautious.



Example:

If the node address should be 42, set switch A to ‘2’ and switch B to ‘4’.

Quick Start Guide

1. Snap the ABC on to the DIN-rail (See 1-2 "DIN Rail Connection")
2. Connect the Profibus cable (See Appendix C-1 "Profibus Connector")
3. Connect the serial sub-network cable (See Appendix C-1 "Sub-network connector")
4. Connect a PC using the PC cable (See Appendix C-2 "PC connector")
5. Connect the power cable and apply power (See Appendix C-1 "Power connector")
6. Start the ABC Config program on the PC (See 4-1 "ABC Config Tool")
   (Normally, the ABC Config software detects the correct serial port, if not select port the menu "Port")
7. Configure the ABC using the ABC Config Tool and download the configuration
8. Configure the sub network device for communication and start it up

Profibus Installation Procedure

Profibus Configuration Tool

Each device on a Profibus-DP network is associated with a .GSD file, containing all necessary information about the device. This file is used by the Profibus configuration tool during configuration of the network. The latest version of this file is available for download at the HMS website, 'www.hms-networks.com'. (The AnyBus Communicator .GSD file is named 'HMS_1803.GSD'.)

It is necessary to import the .GSD file in the Profibus configuration tool in order to incorporate the AnyBus Communicator as a slave in the network. The properties for the AnyBus Communicator module itself must then be configured from the Profibus configuration tool. This includes setting up the node address, input/output data areas and offset address.

- **Node Address**

  The node address in the Profibus configuration tool should be set to match the one selected using the on board configuration switches of the AnyBus Communicator module (See 1-3 "Configuration Switches").

- **Setting up Input / Output Data Areas**

  Input/output data are arranged as logic modules in the Profibus configuration tool. Which modules to use depends on the application. The modules are composed together in the "module list" for the AnyBus Communicator module.

  It is possible to select modules freely to compose the required I/O sizes, see example below.

I/O Bytes required by the application	Modules
4 In + 2 Out	4 In + 2 Out
7 In + 12 Out	4 In + 2 In + 1 In + 8 Out + 4 Out
68 In	64 In + 4 In

- **Offset Address**

  The offset addresses can be chosen freely, however certain restrictions may apply depending on what PLC/Profibus Master is used.

Profibus Network Termination

If the AnyBus Communicator is the last node on a Profibus segment, it is necessary to use a Profibus connector with integrated termination switch.

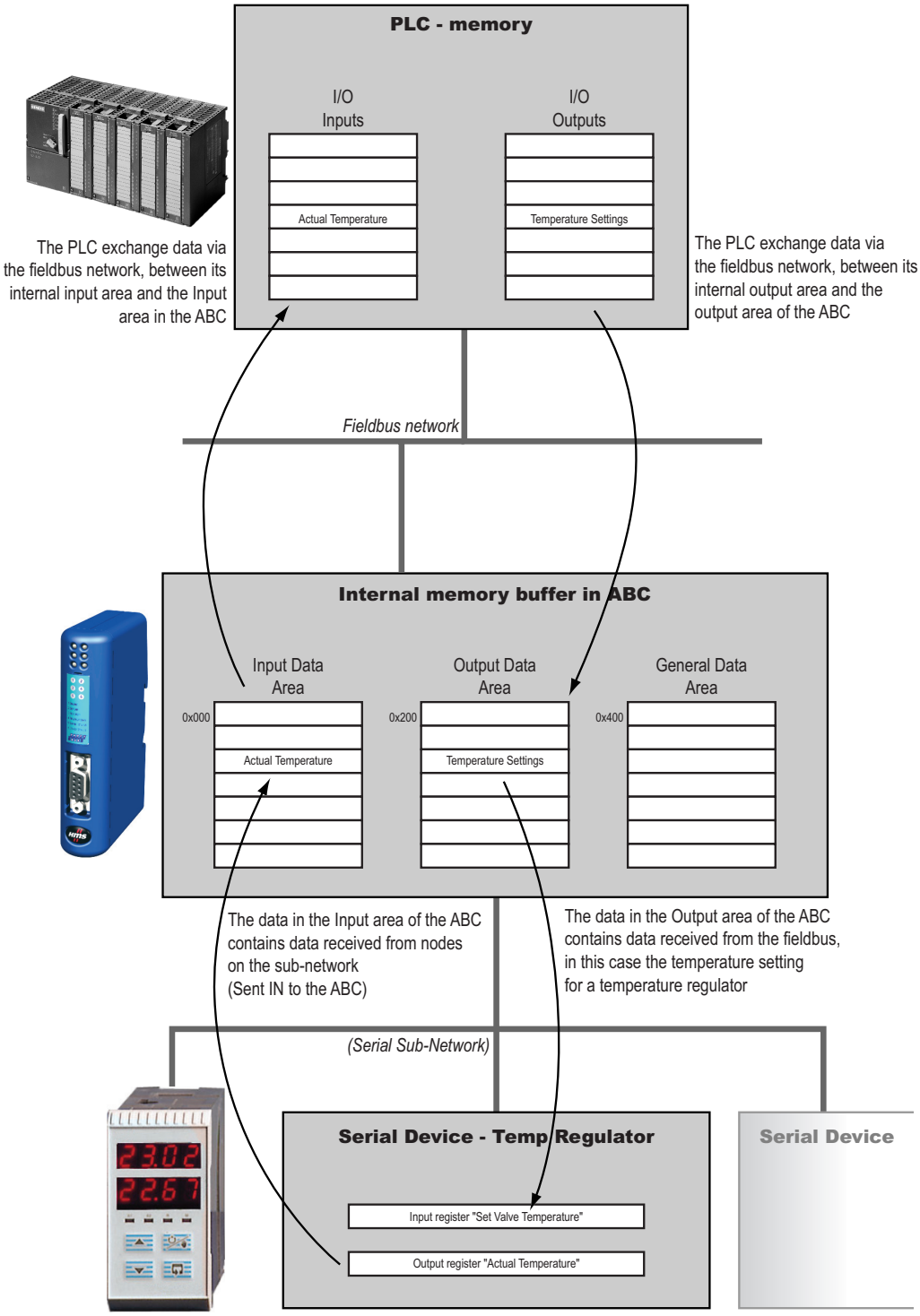
- **The termination switch should be set to 'ON' if...**
  - The ABC module is the last physical node on a network segment
  - No other termination is used at this end of the network
- **The termination switch should be set to 'OFF' if...**
  - There are no other nodes on both side of the ABC module in the network segment

Links

Additional information about the Profibus fieldbus system can be found at 'www.profibus.com'.

Data Exchange

All data from the fieldbus and the sub network is stored in an internal memory buffer. This is a very easy method for data exchange where the fieldbus control system simply reads and writes data to pre-defined memory locations, and the serial sub network also use the same internal memory buffer to read and write data.



Internal Memory Buffer Structure

The internal memory buffer can be seen as a memory space with three different types of data; input data, output data and general data.

- **Input Data**[1]

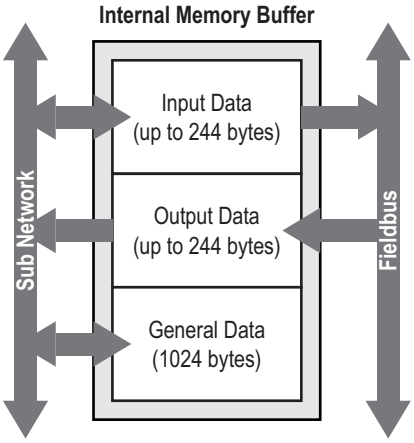
  ..is data that should be sent *to* the fieldbus. The ABC can handle up to 244 bytes of Input Data.

- **Output Data**[1]

  ..is data recieved *from* the fieldbus. The ABC can handle up to 244 bytes of Output Data.

- **General Data**

  This data *cannot* be accessed from the fieldbus, and is used for transfers between nodes on the sub-network, or as a general "scratch pad" for data. The ABC can handle up to 1024 bytes of General Data.



Memory Map

When configuring the sub-network, use the memory locations shown below:

Memory Location:	Contents:	Access:
0x0000 - 0x0001	Status Register	R/W
0x0002 - 0x00F3	Input Data Area	R/W
0x00F4 - 0x01FF	(reserved)	-
0x0200 - 0x0201	Control Register	RO
0x0202 - 0x02F3	Output Data Area	RO
0x02F4 - 0x03FF	(reserved)	-
0x0400 - 0x7FF	General Data Area	R/W

- **Status Register (0x0000 - 0x0001)**

  If enabled, this register occupies the first two bytes in the Input Data Area. For more information, see 3-3 "Status / Control bytes" and 10-1 "Advanced Functions".

- **Input Data Area (0x000 - 0x00F3)**[2]

  This area holds data that should be sent *to* the fieldbus.

- **Control Register (0x0200 - 0x0201)**

  If enabled, these register occupies the first two bytes in the Output Data Area. For more information, see 3-3 "Status / Control bytes" and 10-1 "Advanced Functions".

- **Output Data Area (0x200 - 0x2F3)**[2]

  This area holds data received *from* the fieldbus. Data cannot be written to this area.

1. The total amount if I/O data (Input Data + Output Data) must not exceed 416 bytes.
2. See Status and Control Registers above.

- **General Data Area (0x0400 - 0x7FF)**

  This data *cannot* be accessed from the fieldbus, and should be used for transfers between nodes on the sub-network, or as a general "scratch pad" for data.

ABC Config Tool

The ABC Config Tool is a PC-based configuration software used to describe the protocol and communication properties for a serial network. When the configuration is finished and the functionality is tested, it is possible to send memory allocation information to a printer using the ABC Config Tool.

The ABC Config Tool can also be used for troubleshooting and diagnostic of the ABC and the serial network during runtime.

System requirements

- Pentium 133 MHz or higher
- 10 MB of free space on the hard drive
- 8 MB RAM
- Win95/98/NT/2000/XP
- Internet Explorer 4.01 SP1 or higher

Installation Procedure

There are two different ways of installing the ABC Config Tool; either via the ABC Resource CD-ROM, or via the HMS website, 'www.anybus.com'.

- **AnyBus Communicator resource CD**

  Run 'setup.exe' and follow the on screen instructions

- **From website**

  Download the self-extracting .exe-file from the HMS website, at www.anybus.com, and run it.

The AnyBus Communicator Resource CD is included in the AnyBus Communicator Configuration Pack, (part. no 017620), together with the PC Cable.

Configuration Wizard

When creating a new sub network configuration, the ABC Config Tool provides a choice between starting out with a blank configuration, or using a predefined template, a.k.a a wizard.

- **Configuration Wizard**

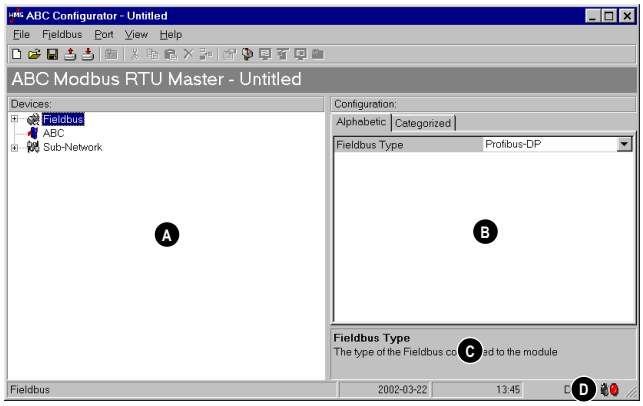
  The wizard option automatically creates a configuration based on information about the sub network devices, i.e the user simply has to "fill in the blanks". Please note that this option is designed to support a particular type of network and cannot be used in all cases.

  (For more information about the Configuration Wizard, see Appendix A-1 "Configuration Wizards")

- **Blank Configuration**

  This option should be used when creating a configuration from scratch, i.e. when the Configuration Wizard does not fit the application. The following chapters will describe the configuration process in detail.

Main Window



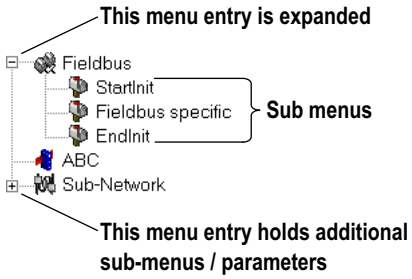
Main Window

A Navigation Window

The navigation window in the ABC Config Tool is the main tool for selecting the different levels of the configuration. There are three main levels in the navigation window, namely Fieldbus, ABC and Sub-network.

Menu entries preceded by a '+' contains more configuration parameters or sub menus. To gain access to these parameters, the entry must be expanded by clicking the '+'.

By right-clicking entries in this window, a popup menu with functions related to this entry will appear. The options in this popup menu is often also available in the menu bar.



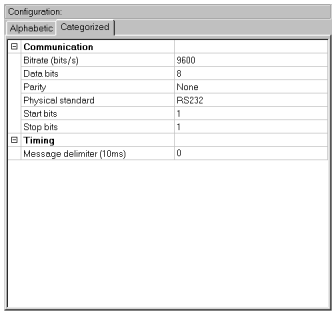
B Parameter Window

The parameters available in this window is different depending on what is selected in the Navigation Window. It consists of a grid with parameter names and, on the same row, a field for editing.

The parameters can be displayed in two modes; Alphabetic and Categorized.

Parameter values are entered either using selection box or by entering a value. Values can be entered either in decimal form (ex: 35) or in hexadecimal form (ex: 0x1A).

If a value is entered in decimal format, it will be converted automatically to the equivalent hexadecimal value.



Parameter Window

C Information Window

In the right bottom corner of the ABC Config Tool application, below the parameter window, lies the information window. It contains descriptions of currently marked parameter instances.

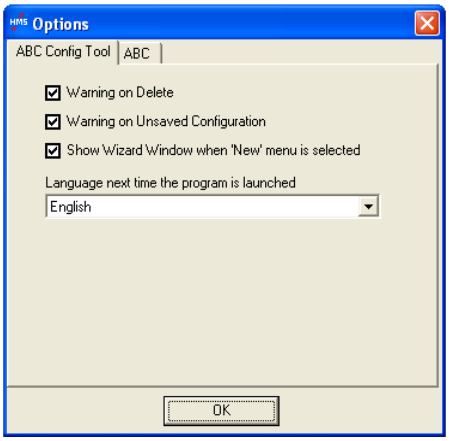
Information Window

D Config Line indicator

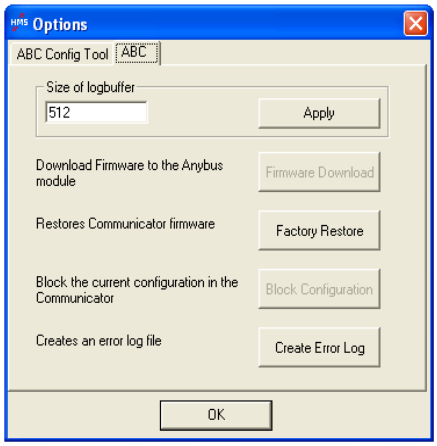
In the lower-right corner of the main window, two lights indicate if a connection has been established between the PC running ABC Config Tool and ABC. Green light - Connection OK, Red light - No connection.

Options

In the main window under tools, select options.

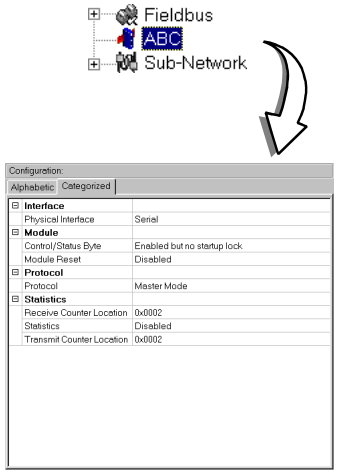


Function	Description
Warning on Delete	When something is to be deleted, a warning window will appear
Warning on unsaved data	When closing the ABC config tool with unsaved data, a warning window will appear
Show Wizard when “New” menu is selected	Each time a new configuration is to be made, the Wizard window will appear
Language next time the program is launched	Select which language the program should use the next time the program is launched



Function	Description
Size of logbuffer	Set the size of the logbuffer(0-512bytes)
Firmware Down-load	Download the firmware to the Anybus-S card. Use with caution
Factory Restore	Restores the software on the ABC-car-rierboard, to it´s original state.
Block Configura-tion	Use with caution. When this button is pressed, the configuation will not be accessible and a new configuration has to be downloaded to the module
Create Error log	Creates an error log file

ABC Configuration



Parameter Window

By selecting 'ABC' in the Navigation window, basic configuration options for the sub-net will appear in the Parameter window.

Physical Interface

Currently, the ABC supports only a serial interface.

The communication settings for the selected interface are available under 'Sub Network', see 13-1 "Serial Interface Settings".

Status / Control bytes

This parameter is used to enable / disable the Status / Control registers, see 20-1 "Control and Status Registers".

- **Enable**

  Enable Status / Control registers. "Data Valid" (Bit 13 in the Control Register) must be set by the fieldbus control system to start the sub network communication.

- **Disable**

  Disable Status / Control registers.

- **Enable but no start up lock**

  The Status / Control registers are enabled, but the fieldbus control system is not required to set the "Data Valid" (bit 13 in the Control Register).

Module Reset

This parameter defines how the module should behave in the event of a fatal error. If Enabled, the module will reset and restart on a fatal error event, and no error will be indicated to the user. If Disabled, the module will halt and indicate an error.

Protocol

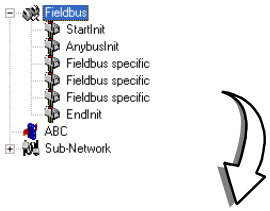
This option is used to configure the communication model used for the sub-network. (This is explained later in this document, see 13-1 "Protocol Configuration")

Statistics

If enabled, the Receive Counter Location indicates the number of valid messages received from the subnet. If enabled, the Transmit Counter Location indicates the number of messages sent to the sub network.

This function is used primarily for debugging purposes.

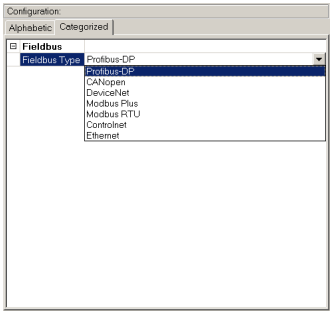
Fieldbus Configuration



During start-up the fieldbus interface of the ABC is initialized to fit the configuration created in the ABC Config Tool.

When several different models of the ABC is used with the same ABC Config Tool installation, the user must verify that the 'Fieldbus' parameter matches to the currently used model of the ABC.

Additionally, it is possible for for advanced users to customize the network interface inside the ABC to meet specific application demands, see 10-5 "Advanced Fieldbus Configuration".



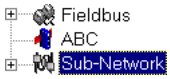
Parameter Window

Sub-network Configuration

When controlling a sub-network with the ABC it is important to understand functions during starting up. If the ABC starts scanning nodes on the sub-network, before data is received from the fieldbus control system (fieldbus master), values of '00' may be transmitted to the nodes before data is updated the first time from the fieldbus.
See 20-4 "I/O-data during startup" for information on how to block transactions until valid data is received.

Serial Interface Settings

To be able to communicate on the sub-network, various communication settings needs to be configured. To gain access to these settings, select 'Sub Network' in the Navigation window.



Parameter	Description	Valid settings
Bit rate	Selects the bit rate used	1200 - 57600
Data bits	Selects the number of data bits used	7, 8
Parity	Selects the parity mode	None, Odd, Even
Physical standard	Selects the physical standard. This setting activates activates the corresponding signals on the sub-net connector.	RS232, RS422, RS485
Start bits	Only one start -bit is supported	1
Stop bits	Either one or two stop -bits can be selected	1, 2

Protocol Configuration

In order to be able to communicate on the sub-network, the ABC must be supplied with a description of the required sub-net protocol. To accomplish this, the ABC Config software features a flexible protocol-programming system, allowing the ABC to interpret and exchange data with almost any serial device on the sub-network.

Communication model

The ABC supports two communication models; Generic Data Mode, and Master Mode. (This option appears in the Parameter window upon selecting 'ABC' in the Navigation window.)

Note that this setting is used to describe the relationship between the sub-net nodes (including the ABC), not the exact protocol used to transmit data.

- **Generic Data Mode**

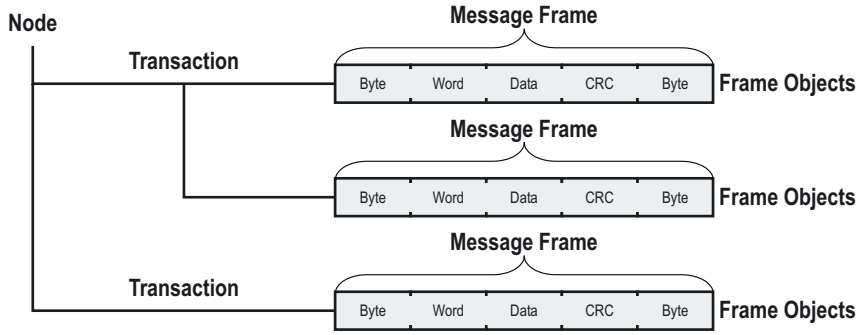
  In this mode, there is no Master / Slave relationship between the ABC and the sub-net nodes. It is intended for Produce / Consume oriented protocols. This mode is described in greater detail later in this document, see 15-1 "Generic Data Mode".

- **Master Mode**

  In this mode, the ABC is setup to use a Modbus RTU protocol or similar, and implements a Modbus Master for data exchange between the fieldbus and one or more devices on the sub-network. This mode is explained in greater detail in chapter 14-1 "Master Mode".

Protocol Building Blocks

Below is a description of the building blocks used to describe the sub-net protocol. The exact structure of these building blocks varies depending on the selected communication model.



- **Node**

  In the ABC Configurator, a node holds all transactions and parameters for a particular device on the sub network.

- **Transaction**

  Transactions contains messages to be transmitted on the sub-network. A transaction consists of one or more Message Frames (see below), and has a few general parameters to specify how and when the transaction should be used on the sub-network.

- **Commands**

  A command is a pre-defined transaction that has been stored in a list in the ABC Configuration software. This improves readability in the ABC Configuration software, as well as simplifies common operations by allowing transactions to be stored and re-used.

- **Message Frame**

  The message frame contains a description of what is actually transmitted on the sub-network and consists of frame objects, see below.

- **Frame Object**

  Frame objects are used to compose a message frame. A frame object can be a fixed value, a dynamic value retrieved from a specified memory location in the ABC, a string etc.

Generic Data Mode

Introduction

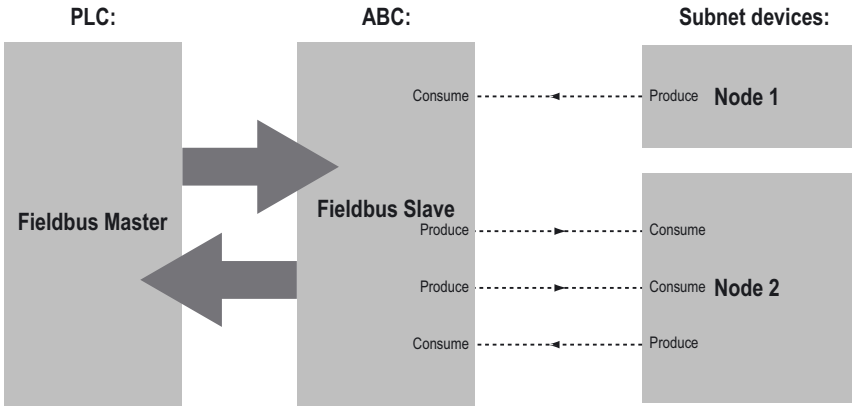
In Generic Data Mode, the ABC is designed to communicate with the following types of equipment:

- **Data Producers**
  Equipment that sends byte strings
- **Data Consumers**
  Equipment that receives byte strings
- **Data Producers and Consumers**
  Equipment that both sends and receives byte strings

In Generic Data Mode, there is no master-slave relationship between the sub-net nodes and the ABC. Any node on the sub-net, including the ABC, can spontaneously produce or consume a message; A node does not have to reply to a message, nor does it have to wait for a query to send one.

In the example below, the ABC "Consumes" data that is "Produced" by a node on the sub-network. This "Consumed" data can then be forwarded to the fieldbus master.

This also works the other way around; the ABC receives a data telegram from the fieldbus master, and use this data to "Produce" a message on the sub-network to be "Consumed" by a node.

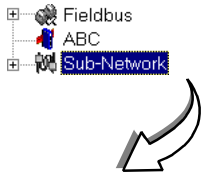


It is to some degree possible to implement a Query / Response based protocol in this mode, however, Master Mode is better suited for this. (See 14-1 "Master Mode")

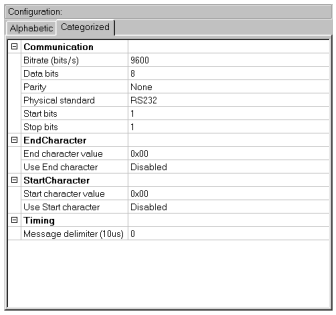
Generic Data Mode can be used in both full duplex point-to-point systems (RS232 and RS422) as well as in half duplex multi-drop systems (RS485).

Note: The ABC will not check any bus access algorithms when operating in this mode; This must be handled by the fieldbus control system, i.e the PLC.

Basic Settings



Select 'Sub Network' in the Navigation window to gain access to basic settings in the Parameter window.



Parameter Window

Communication

(See 13-1 "Serial Interface Settings")

Start and End Character

Start and end characters are used to indicate the beginning and end of a message. For example, a message may be initiated with <ESC> and terminated with <LF>. In this case, the Start character would be 0x1B (ASCII code for <ESC>) and the End character 0x0A (ASCII code for <LF>)

Parameter	Description	Valid settings
End Character Value	End character for the message, ASCII	0x00 - 0xFF
Use End Character	Determines if the End character should be used or not	Enable / Disable
Start Character Value	Start character for the message, ASCII	0x00 - 0xFF
Use Start Character	Determines if the Start character should be used or not	Enable / Disable

Message Delimiter

The Message delimiter is the timeout time when receiving a message on the sub-network. For most protocols the recommended timeout setting is at least 10 times the response time of a node.

- For Consume objects this option tells the ABC how long after the last byte is received it has to wait before a complete message is in.
- For Produce objects, this instructs the ABC how long it should wait before a new message is sent.

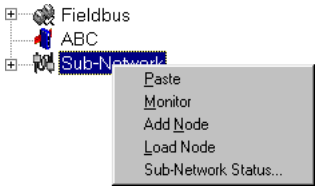
Nodes

A node in the ABC Configuration software represents a device on the sub-network. In Generic Data Mode, a node can carry up to 50 transactions.

Sub-Network Menu

(Right-click 'Sub Network' in the Navigation window to gain access to these options)

Function	Description
Paste	Paste a node from the clipboard
Subnet Monitor	Launches the subnet monitor, see 18-1 "Sub Network Monitor".
Add Node	Adds a node
Load Node	Loads a node previously saved using the 'Save Node'-function, see 'Node Menu' below.
Sub-Network Status..	Displays status / diagnostic information about the sub network



Node Settings

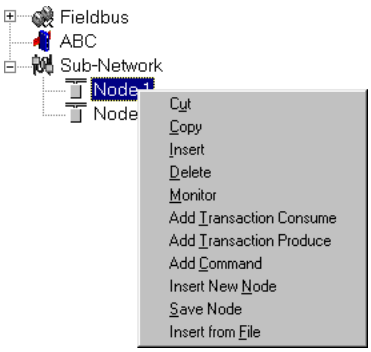
(Select a node in the Navigation window to gain access to these parameters in the Parameter Window)

Function	Description
Slave Address	This setting shall be set to match the node address setting of the destination device.
Name	Node Name. This name will appear in the navigation window.

Node Menu

(Right-click on a node in the Navigation window to gain access to these options)

Function	Description
Cut	Cuts a node to the clipboard
Copy	Copies a node to the clipboard
Insert	Insert a node from the clipboard
Delete	Deletes a node and its configuration
Node Monitor	Launches the node monitor
Add Transaction Consume	Adds a Consume transaction
Add transaction Produce	Adds a Produce transaction
Add command	Adds a pre-defined transaction
Insert new node	Inserts a new node above the currently selected node
Save node	Saves the selected node to disc
Insert from file	Inserts a previously saved node above the currently selected node.



Transactions

In Generic Data Mode, there are two types of transactions:

- **Transaction Consume**

  This transaction is used to receive or "consume" data from the sub-net. By using a Consume transaction, data can be forwarded from the sub-network to the fieldbus.

- **Transaction Produce**

  This transaction is used to transmit or "produce" data on the sub-network without waiting for a response. By using a Produce transaction, data can be forwarded from the fieldbus to the sub-network.

Transaction Consume Parameters

(To gain access to these parameters, select a Consume Transaction in the Navigation window)

Parameter	Description
Offline options for sub-network	The action to take for this transaction if the sub-network goes off-line. This action affects the data that is reported to the fieldbus master. • **Clear** The data is cleared (0) on the fieldbus if the sub-network goes offline • **Freeze** The data is frozen on the fieldbus if the sub-network goes offline
Offline timeout time (10ms)	The Offline Timeout value is the maximum allowed time between two incoming messages in steps of 10ms. If this time is exceeded, the sub network will be considered to be off line. A value of 0 disables this feature, i.e. the sub network can never go off line.
Trigger byte	The trigger byte is used to indicate to the fieldbus control system that a valid telegram has been consumed on the sub-network. The control system should then read the data area connected to the trigger byte. The value of this byte is increased by 1 whenever a valid sub-network telegram has been consumed and interpreted by the ABC. • **Enable** Enables the trigger byte. The memory location of the trigger byte must be specified in the 'Trigger byte address', see below. • **Disable** Enables the trigger byte.
Trigger byte address	The location in the internal memory buffer that this transaction uses for updates on trigger byte changes This memory location should be monitored by the fieldbus control system. If the contents has been updated, e.g the value has been incremented, a valid sub-network telegram has been consumed by the ABC and new data is available in the internal memory buffer.

Transaction Produce Parameters

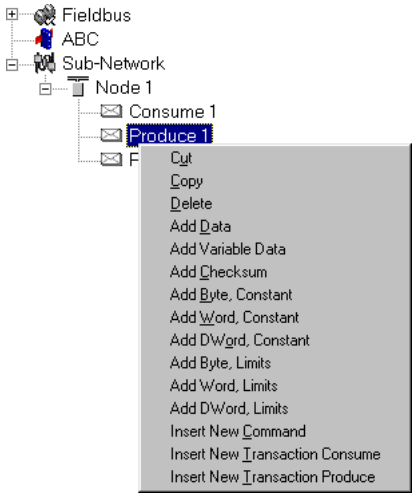
(To gain access to these parameters, select a Produce Transaction in the Navigation window)

Parameter	Description
Offline options for fieldbus	The action to take for this transaction if the fieldbus goes off-line. This action affects the data that is sent on the sub-network. • **Clear** The data is cleared (0) on the sub-network if the fieldbus goes offline • **Freeze** The data is frozen on the sub-network if the fieldbus goes offline • **NoScanning** Stop sub-net scanning for this transaction if the fieldbus goes offline
Update mode	The update mode for the transaction • **Cyclically** The data is transmitted cyclically. The update frequency is determined by the Update Time • **On data change** The data is updated when any data in the message has changed • **Single shot** The data is transmitted once at startup • **Change of state on trigger** The data is transmitted when the trigger byte has changed. (See 'Trigger byte address' below)
Update time (10ms)	This value determines how often the data is updated on the sub-network. A value of 0x000A equals 100ms. Valid settings range from 0x0000 to 0xFFFF.
Trigger byte address	This parameter specifies location of the trigger byte in the internal memory buffer. If 'Update mode' is set to 'Change of state on trigger', the memory location specified by this parameter is monitored by the ABC. Whenever the trigger byte is updated, the ABC will produce the transaction on the sub-network. This way, the fieldbus control system can when required instruct the ABC to produce a specific transaction on the sub-network by updating it's trigger byte. The trigger byte should be incremented by one for each activation. This parameter has no affect unless the 'Update mode' parameter is set to 'Change of state on trigger'.

Produce / Consume Menu

(Right-click a Produce or Consume transaction in the Navigation window to gain access to these options)

Action	Description
Cut	Cuts a transaction to the clipboard.
Copy	Copies a transaction to the clipboard.
Delete	Deletes a transaction
Edit frame	Launches the Frame Editor
Add Data	Adds a fixed length data object
Add Variable Data	Adds a variable length data object to
Add Checksum	Adds a checksum object to the frame
Add Byte, Constant	Adds a one-byte object to the frame
Add Word, Constant	Adds a 2-byte object to the frame
Add DWord, Constant	Adds a 4-byte object to the frame
Add Byte, Limits	Adds a one-byte interval to the frame
Add Word, Limits	Adds a 2-byte interval to the frame
Add DWord, Limits	Adds a 4-byte interval to the frame
Insert New Command	Inserts a predefined transaction
Insert New Transaction Consume	Inserts a Consume transaction
Insert New Transaction Produce	Inserts a Produce transaction

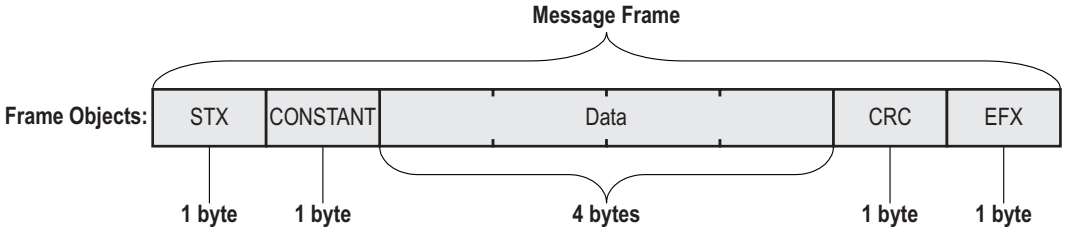


See 15-7 “Frame Objects” for a description of the different frame objects

Frame Objects

The Produce and Consume message frames are built up of frame objects, that when combined makes a complete serial telegram.

Example:



Constants

Constants are objects built up of pre-defined data. The contents of these objects have a fixed value defined in the ABC Config Tool.

In a Consume transaction, the ABC will check if the received byte/word/doubleword match this value. If not, the ABC will discard the message.

There are 3 types of Constants:

- **Byte**
  8 bit fixed value
- **Word**
  16 bit fixed value
- **Dword**
  32 bit fixed value

Parameter	Description
Value	Constant value.

Checksum Object

Most serial protocols has some way of verifying that the data has not been corrupted during transfer. The Checksum object is an object that can be used to calculate the checksum for a message frame.

Parameter	Description
Error Check Start	Offset in the message frame to start checksum calculations on
Error Check Type	The type of error checking algorithm to use - CRC, LRC or XOR • **CRC - Cyclic Redundancy Check** • **LRC - Longitudinal Redundancy Check** • **XOR - Logical XOR**

Limit / Interval Objects

Interval objects are objects with a pre-defined range. The range of these object is defined in the ABC Config Tool., and must always be within the previously defined range in the serial telegram.

In a Consume transaction, the ABC will check if the received byte/word/doubleword is "inside" the defined boundaries. If not, the ABC will discard the message.

There are 3 types of interval objects:

- **Byte**
  8 bit interval
- **Word**
  16 bit interval
- **Dword**
  32 bit interval

Parameter	Description
Maximum Value	This is the largest allowed value for the range. Range: 0x00 to 0xFFh for Byte, 0xFFFF for Word, 0xFFFFFFFF for DWord. (This value must be larger than the Minimum value)
Minimum Value	This is the smallest allowed value for the range. Range: 0x00 to 0xFFh for Byte, 0xFFFF for Word, 0xFFFFFFFF for DWord. (This value must be less than the Maximum value)

Data Object

The Data Object is used for data exchange between the fieldbus master and the sub-network.

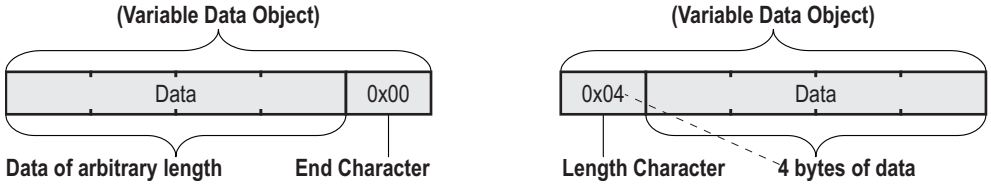
- **Data Objects in Produce Transactions:**
  This object is used to forward data from the sub-network to the fieldbus.
- **Data Objects in Consume Transactions:**
  This object is used to forward data from the fieldbus to the sub-network

Parameter	Description
Byte Swapping	• **No Swapping** No swapping is performed on the data • **Swap 2 bytes** This means that the 2 bytes change places, i.e A, B, C, D becomes B, A, D, C • **Swap 4 bytes** This means A, B, C, D becomes D, C, B, A
Data Length	The length of the data, in bytes
Data Location	The offset in the Internal memory buffer where the data should be read from / written to

Variable Data Object

The Variable Data Object is similar to the Data Object, but has no predefined length. Instead, a length character or end character is used to indicate the length of the datafield.

As with the Data Object, this object is used to transfer data between the sub-network and fieldbus.



For Produce Transactions, the End / Length character must be supplied by the fieldbus control system. For Consume Transactions, the End / Length character is generated by the ABC.

The End / Length character is always visible in the internal memory buffer. Depending on the settings below, it may or may not be visible on the sub-network.

Note: Only one Variable Data Object is permitted for each transaction.

Parameter	Description
Byte Swapping	• **No Swapping** No swapping is performed on the data • **Swap 2 bytes** This means that the 2 bytes change places, i.e A, B, C, D becomes B, A, D, C • **Swap 4 bytes** This means A, B, C, D becomes D, C, B, A
Fill un-used bytes	(This field is only relevant for Consume transactions) • **Enabled** Fill unused data with the value specified in 'Filler byte' • **Disabled** Don't fill
Filler byte	Fill value, see 'Fill un-used bytes' above. (This field is only relevant for Consume transactions)
Data Location	The offset in the internal memory buffer where the data should be read from / written to
Object Delimiter	• **Length Character** Length character is visible in the internal memory buffer but *not* on the sub network • **Length Character Visible** The length character is visible both in the internal memory buffer *and* on the sub network. • **End Character** The end character is visible in the internal memory buffer but *not* on the sub network. • **End Character Visible** The end character is visible both in the internal memory buffer *and* on the sub-network • **No Character** The data is copied "as is" to the internal memory buffer, i.e no end character is generated by the ABC (This options is only relevant for Consume Transactions)
End Character Value	Value of the End Character. (This is only be used when the Object Delimiter is set to 'End Character' or 'End Character Visible'
Maximum Data Length	The maximum allowed length of the variable data object.

Master Mode

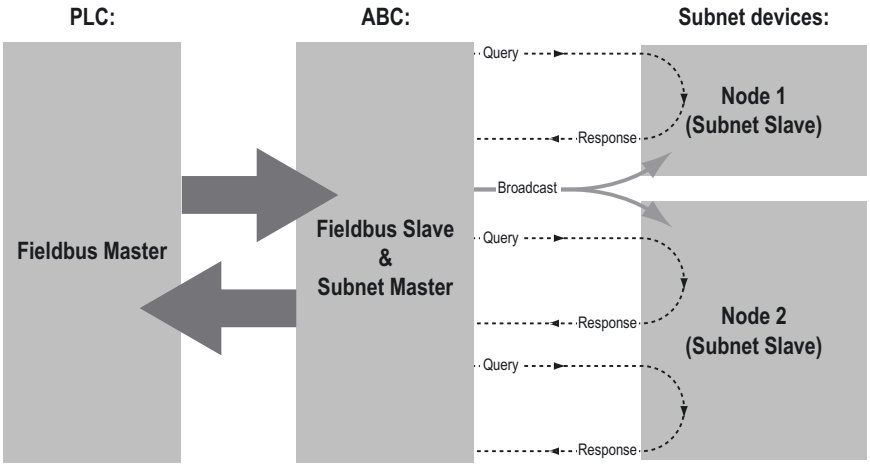
Introduction

In Master Mode, the ABC is configured to run as a master on the sub-network, using a scan-list for communication with the slave devices. The scan-list is created using the ABC Config Tool and can consist of multiple nodes with multiple transactions.

In Master Mode, communication between the ABC and the sub-net nodes is based on transactions with a Query / Response architecture. The ABC sends out a Query on the sub-network, and the addressed node is expected to send a Response to this Query. Slave nodes are not allowed to Respond without getting a Query first.

An exception to this is the broadcaster functionality. Most protocols offer some way of accessing all nodes on the network. In the ABC, this is called a 'Broadcaster'. The Broadcaster can transmit messages to all nodes on the sub-network, but does not expect a response.

In Modbus, it is possible to broadcast a message to all nodes by sending a message to node address 0. The Modbus slaves will receive the message, but not Respond to it.

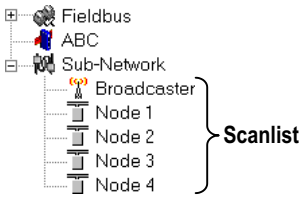


The base in Master Mode is the pre-configured Modbus RTU commands, acting as a Modbus RTU master. With Modbus RTU, each transaction is substituted with a pre-defined command that can be selected from a list of available commands.

It is still possible, though, to define custom message frames by creating a transaction instead of selecting a pre-defined command. A command is actually a transaction that has been defined in advance and stored in a list.

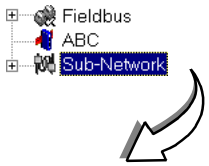
The Scan list

Once the configuration has been made and downloaded to the ABC, the ABC's firmware searches the scan-list, using the defined transactions for communication with the slave-devices.

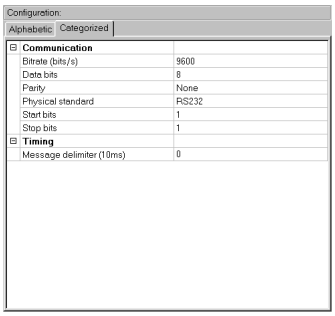


Each node in the scan-list represents a slave device on the sub-network. In the ABC Config Tool, each node is given a specific name and assigned an address in standard Modbus RTU commands. The address must match the internal setting on the slave device.

Basic Settings



Select 'Sub Network' in the Navigation window to gain access to basic settings in the Parameter window.



Parameter Window

Communication

(See 13-1 "Serial Interface Settings")

Message Delimiter

The value entered here is the minimum time in steps of 10ms, separating the messages. (According to the Modbus specification, the Message Delimiter has a default setting of 3.5 characters.)

If this value is set to '0', the ABC will use the Modbus standard 3,5 character message delimiter. The time in ms is then dependent on the selected baudrate, but this is all handled by the ABC.

Note: Due to its big impact on subnet functionality, use caution when changing this parameter.

Nodes

A node in the ABC Configuration software represents a device on the sub-network. In it's simplest form, a Node contains of a single transaction, that consists of a Query and a Response.

Node Parameters

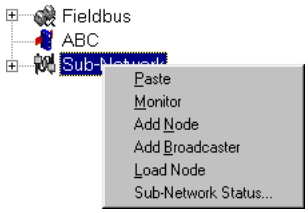
(To gain access to these parameters, select the desired node in the navigation window)

Parameter	Description
Slave Address	This setting shall be set to match the node address setting of the destination device.
Name	Node Name. This name will appear in the navigation window.

Sub-Network Menu

(Right-click "Sub Network" in the Navigation window to gain access to these functions)

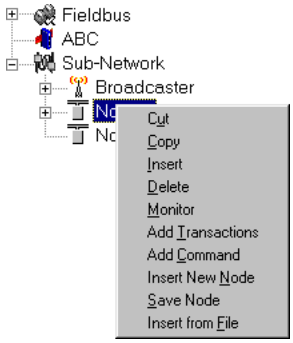
Function	Description
Paste	Paste a node from the clipboard
Monitor	Launches the subnet monitor, see 18-1 "Sub Network Monitor"
Add Node	Adds a node to the scanlist
Add Broadcaster	Adds a broadcaster node to the scanlist
Load Node	Loads a node previously saved using "Save Node" from the Node menu (see below)
Sub-Network Status..	Displays status / diagnostic information about the sub network



Node Menu

(Right-click on a node in the Navigation window to gain access to these functions)

Function	Description
Cut	Cuts a node to the clipboard
Copy	Copies a node to the clipboard
Insert	Insert a node from the clipboard
Delete	Deletes a node and its configuration from the scan-list
Monitor	Activates the node monitor
Add transactions	Adds a generic command to the scan-list. This command is fully configurable by the user
Add command	Adds a pre-defined protocol specific command to the scan-list. The list of commands are supplied with the ABC and cannot be changed
Insert new node	Inserts a new node above the currently selected node
Save node	Saves the selected node to disc
Insert from file	Inserts a previously saved node above the currently selected node.



Transactions

In Master Mode, each transaction consists two instances; (a Query and a Response) unless it's a Broadcaster.

- **Query Transaction**

  In Master Mode, a "Query" is defined as a telegram sent from the master (ABC) to the slave (Node).

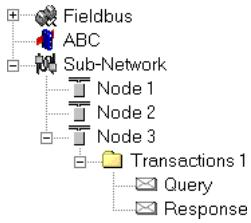
- **Response Transaction**

  In Master Mode, a "response" is defined as a reply from a slave (Node) to a previous "query" from the master (ABC).

  The response defines the expected answer from the slave device. If the answer does not match the pre-defined response the ABC will try to re-send the query according to the parameters specified in the query.

Example:

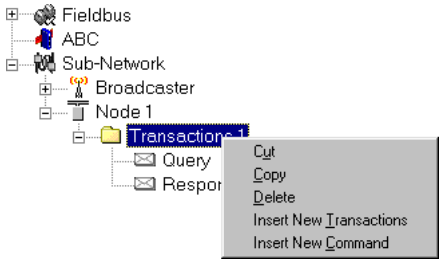
In this example, Node 3 holds a transaction that consists of a Query and a Response.



Transaction Menu

(Right-click on a transaction in the Navigation window to gain access to these functions)

Function	Description
Cut	Cuts a transaction to the clipboard.
Copy	Copies a transaction to the clipboard.
Insert	Inserts a Cut or Copied transaction.
Delete	Deletes the transaction
Insert New Transactions	Inserts a new transaction.
Insert New Command	Inserts a command from a pre-defined list of commands.



Query Parameters

(To gain access to these parameters, select a Query in the Navigation window)

Parameter	Description
Minimum time between broadcasts (10ms)	The value entered here is only valid if a broadcast command is specified in the scan-list and the value specifies how long the ABC should wait after the broadcast was sent until the next command in the scan-list will be sent. This time should be selected such that all slave-devices connected to the ABC have time to finish the handling of the broadcast. The unit is milliseconds (ms) and the entered value is multiplied by 10, which means that the shortest time is 10 ms.
Offline options for fieldbus	This parameter defines the behavior of the ABC in case the fieldbus network goes off-line and the selection affects the data that is sent out the sub-network • **Clear** - All data destined for the slave-devices is cleared (set to 0) • **Freeze** - All data destined for the slave-device is frozen • **NoScanning** -The updating of the sub-network is stopped
Offline options for sub-network	This parameter defines the behavior of the ABC in case the sub-network goes off-line and the selection affects the data that is reported to the fieldbus master. • **Clear** - All data destined for the fieldbus-master is cleared (set to 0) • **Freeze** - All data destined Note: Offline options for subnetworks are configured separately for each transaction
Reconnect time (10ms)	This parameter specifies how long the ABC should wait before trying to re-connect a disconnected node. A node gets disconnected if the max number of retries is reached. The unit is milliseconds (ms) and the entered value is multiplied by 10, which means that the shortest time is 10 ms.
Retries	This parameter specifies how many times a time-out can occur in sequence before the slave is disconnected.
Timeout time (10ms)	This parameter specifies the time the ABC waits for a response from the slave-device. If this time is exceeded the ABC re-sends the command until the “retries” parameter value is reached. The unit is milliseconds (ms) and the entered value is multiplied by 10, which means that the shortest time is 10 ms.
Trigger byte address	This parameter specifies location in the internal memory buffer where the trigger byte is located. In ABC a trigger byte is implemented to support non-cyclic data that means that the fieldbus master has the ability to notify the ABC when it should send a specific command to a slave. To use this functionality correctly the fieldbus master should update the data area associated with the trigger byte, and then update the trigger byte. The trigger byte should be incremented by one for activation. This parameter has no affect unless the “Update mode” parameter is set to “Change of state on trigger”.
Update mode	This parameter is used to specify when the command should be sent to the slave. The following modes are possible: • **Cyclically** - The command is sent to the slave at the time-interval specified in the “Update time” parameter. • **On data change** -The command is sent to the slave when the data-area connected to this command changes. • **Single shot** - The command is sent to the slave once at start-up. • **Change of state on trigger** - The command is sent to the slave when the trigger byte value is changed
Update time (10ms)	This parameter specifies with what frequency this command will be sent. The unit is milliseconds (ms) and the entered value is multiplied by 10, which means that the shortest time is 10 ms.

Response Parameters

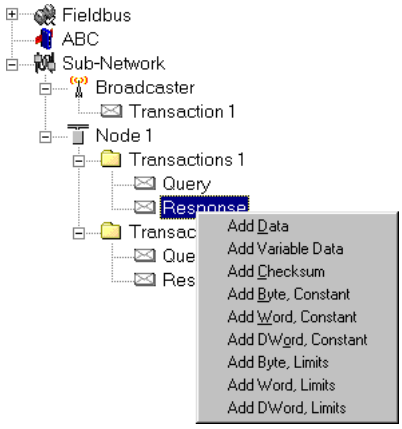
(To gain access to these parameters, select a Response in the Navigation window)

Parameter	Description
Trigger byte	This parameter disables and enables the trigger functionality for the response. If the "trigger byte" is enabled then the ABC will increase the byte at the "trigger byte address" by one when the ABC receives new data from the sub-network. This will notify the fieldbus master of updated data
Trigger byte address	This parameter is used to specify the address in the internal memory buffer where the trigger byte is located. Valid settings range from 0x202 to 0x3FF

Query / Response Menu (also Broadcaster Transactions)

(Right-click a Query/Response in the Navigation window to gain access to these functions)

Function	Description
Edit Frame	Launches the Frame editor
Add Byte, Constant	Adds a one-byte object to the frame.
Add Word, Constant	Adds a 2-byte object to the frame.
Add DWord, Constant	Adds a 4-byte object to the frame.
Add Checksum	Adds an error check object to the frame.
Add Data	Adds a fixed length data object to the frame.
Add Variable Data	Adds a variable length data object to the frame



Note: If the selected Query / Response does not contain any frame objects, the 'Edit Frame' function will not be available

The frame editor is further described in the section Frame editor. By using these different selections it is possible to define custom data-frames that the ABC will send out on the sub-network.

Example:

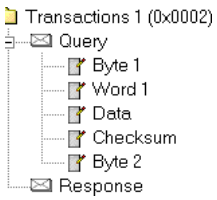
To add a Query like the one below to the scan list...

STX (1 byte)	Length (2 bytes)	Data (8 bytes)	Checksum[a]	ETX (1 byte)

a. The length depends on Checksum type, see 14-9 "Checksum Object".

...the following steps are needed:

Right-click on the "query" and select in order: Add Byte, Add Word, Add Data, Add Error check and Add Byte. The "query" now looks like this:



Frame Objects

All Query and Response messages are built up of frame objects, that when combined makes a complete serial telegram.

Important to note is that these frame objects are not Modbus specific; Modbus is only used as an example.

The only things that are modbus specific are the names “query” and “response” and the fact that all transactions are of “query-response” (question-answer) type.

Constants

Constants are objects built up of pre-defined data. The contents of these objects have a fixed value defined in the ABC Config Tool.

There are 3 types of fixed objects:

- **Byte**
  8 bit fixed value
- **Word**
  16 bit fixed value
- **Dword**
  32 bit fixed value

Parameter	Description
Value	Constant value.

Data Object

The Data Object is used for data exchange between the fieldbus master and the sub-network.

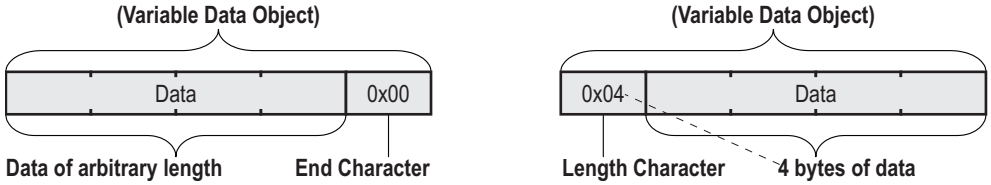
- **Data Objects in Query Transactions:**
  This object is used to forward data from the sub-network to the fieldbus.
- **Data Objects in Response Transactions:**
  This object is used to forward data from the fieldbus to the sub-network.

Parameter	Description
Byte Swapping	• **No Swapping** No swapping is performed on the data • **Swap 2 bytes** This means that the 2 bytes change places, i.e A, B becomes B, A • **Swap 4 bytes** This means A, B, C, D becomes D, C, B, A
Data Length	The length of the data, in bytes
Data Location	The offset in the Internal memory buffer where the data should be read from / written to

Variable Data Object

The Variable Data Object is similar to the Data Object, but has no predefined length. Instead, a length character or end character is used to indicate the length of the datafield.

As with the Data Object, this object is used to transfer data between the sub-network and fieldbus.



For Produce Transactions, the End / Length character must be supplied by the fieldbus control system. For Consume Transactions, the End / Length character is generated by the ABC.

The End / Length character is always visible in the internal memory buffer. Depending on the settings below, it may or may not be visible on the sub-network.

Note: Only one Variable Data Object is permitted for each transaction.

Parameter	Description
Byte Swapping	• **No Swapping** No swapping is performed on the data • **Swap 2 bytes** This means that the 2 bytes change places, i.e A, B, C, D becomes B, A, D, C • **Swap 4 bytes** This means A, B, C, D becomes D, C, B, A
Fill un-used bytes	(This field is only relevant for Response transactions) • **Enabled** Fill unused data with the value specified in 'Filler byte' • **Disabled** Don't fill
Filler byte	Fill value, see 'Fill un-used bytes' above. (This field is only relevant for Response transactions)
Data Location	The offset in the internal memory buffer where the data should be read from / written to
Object Delimiter	• **Length Character** Length character is visible in the internal memory buffer but *not* on the sub network • **Length Character Visible** The length character is visible both in the internal memory buffer *and* on the sub network. • **End Character** The end character is visible in the internal memory buffer but *not* on the sub network. • **End Character Visible** The end character is visible both in the internal memory buffer *and* on the sub-network • **No Character** The data is copied "as is" to the internal memory buffer, i.e no end character is generated by the ABC (This options is only relevant for Response Transactions)
End Character Value	Value of the End Character. (This is only be used when the Object Delimiter is set to 'End Character' or 'End Character Visible'
Maximum Data Length	The maximum allowed length of the variable data object.

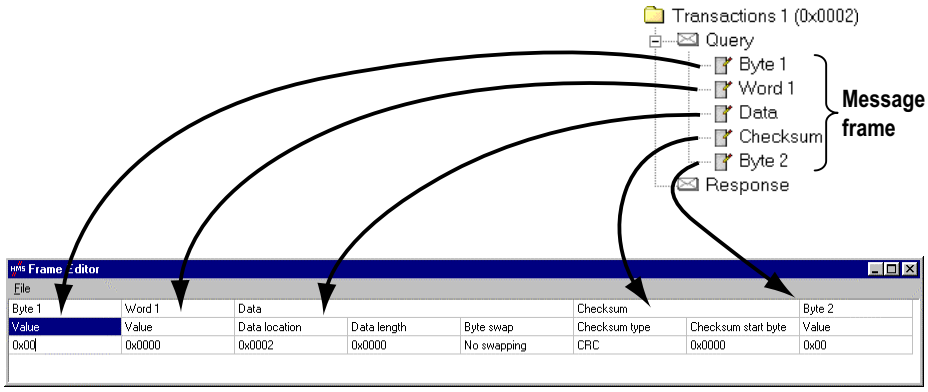
Checksum Object

Most serial protocols has some way of verifying that the data has not been corrupted during transfer. The Checksum object is an object that can be used to calculate the checksum for a message frame.

Parameter	Description
Error Check Start	Offset in the message frame to start checksum calculations on
Error Check Type	The type of error checking algorithm to use - CRC, LRC or XOR • **CRC - Cyclic Redundancy Check** • **LRC - Longitudinal Redundancy Check** • **XOR - Logical XOR**

Frame editor

The frame editor makes it easier to add specific custom commands. The same parameters are available in both the frame editor and the parameter window, but in the frame editor presents the message frames in a more visual manner than the navigation / parameter window.

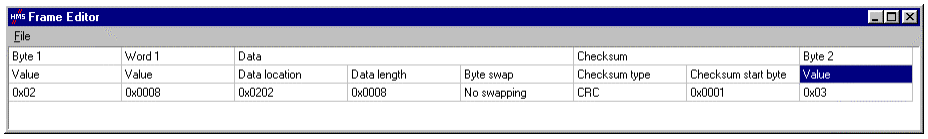


Frame editor

Note: The example below uses Master Mode, but the procedure is similar in General Data Mode.

Example:

The frame looks like this:



Frame editor

The first byte holds the STX (0x02) followed by two bytes holding the length (8 in this case). The next 8 bytes are data and since this is a "query" command, the data is to be sent to the slave device and therefore it is to be fetched from the out area which starts at 0x202.

This command will allocate 8 bytes of output data in the out area and no byte swapping will occur. The data is followed by a 2 byte CRC error check field, and the CRC calculation starts with the second byte in the frame (i.e. STX is not included in the CRC calculation). The last byte is the ETX (0x03).

The same steps are required for the response frame. If the response holds data, it should be allocated in the input area that starts at address 0x002.

To apply the changes, select File | Apply Changes. To exit without saving, select File | Exit.

Command editor

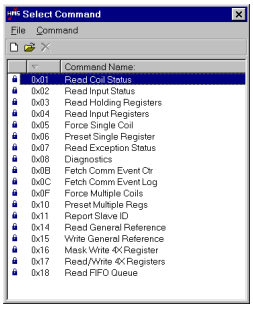
General

The Command Editor makes it possible to add custom commands to the ABC. The Command Editor is protocol dependent in that sense that certain frame objects cannot be deleted.

Note: The example in this chapter uses Master Mode. The procedure is similar in General Data Mode, but without the limitations of the Modbus RTU protocol.

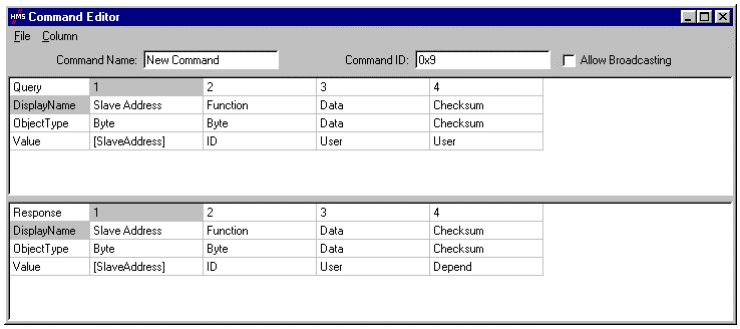
To open the command editor, right click a node and select 'Add Command'. A list of predefined commands will appear.

To add a new command to the command list, select 'Add Command' in the 'Command' menu. To edit a previously defined command, highlight the command in the command list, and select 'Edit Command' in the 'Command' menu.



Select Command

The following window pops up upon selecting 'Edit Command' or 'Add Command'



Command Editor

Specifying a new command (Master Mode)

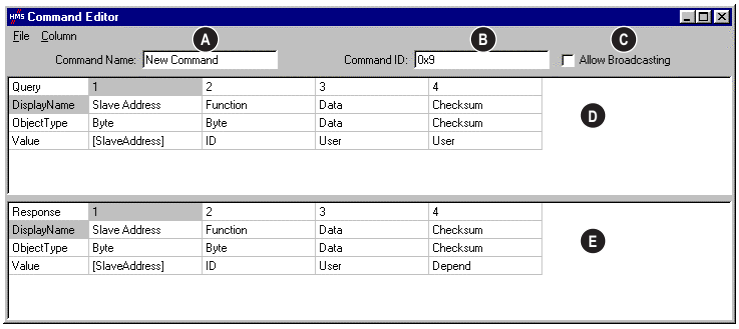
(Select 'Add Command' as described earlier.)

This example is taken from a Modbus RTU implementation, which means that the frame will always consist of one byte for slave address, one byte for function code and two bytes for CRC. Furthermore, each command always consists of a Query and a Response.

The Modbus RTU specific frame objects are already in place and a data object is inserted between the function code and the CRC. These objects cannot be moved or deleted, however it possible to add objects between the function code and the CRC.

First, enter a name for the command in the Command Name field (A) and an identifier in the Command ID field (B).

If the command is allowed to be broadcasted on the sub-network, check the Allow Broadcasting checkbox (C).

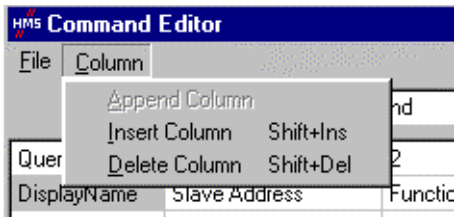


Command Editor

Query (D)

Query	1	2	3	4
	This column specifies the SlaveAddress.	This column specifies the Function Code in the Modbus message	(See below)	This column specifies the Error Check field.
DisplayName	Slave Address	Function	Data	Checksum
	This name is protocol specific, and cannot be altered.	This name is protocol specific, and should not be changed.	(See below)	
Object Type	Byte	Byte	Data	Checksum
	Modbus defines this object as a byte.	Modbus defines this object as a byte.	(See below)	
Value	[SlaveAddress]	ID	User	User
	This value is linked to the actual 'SlaveAddress' parameter in the parameter window.	This value is linked to the Command ID field.	(See below)	This value is linked to "User" which means that this object is determined by the user at configuration time by selecting the Error Check object in the parameter window.

It is not possible to alter the contents of columns 1, 2 and 4, as this is a predefined command. However, on column three there are two possible actions available, Insert Column and Delete Column. These actions are available in the Columns menu.



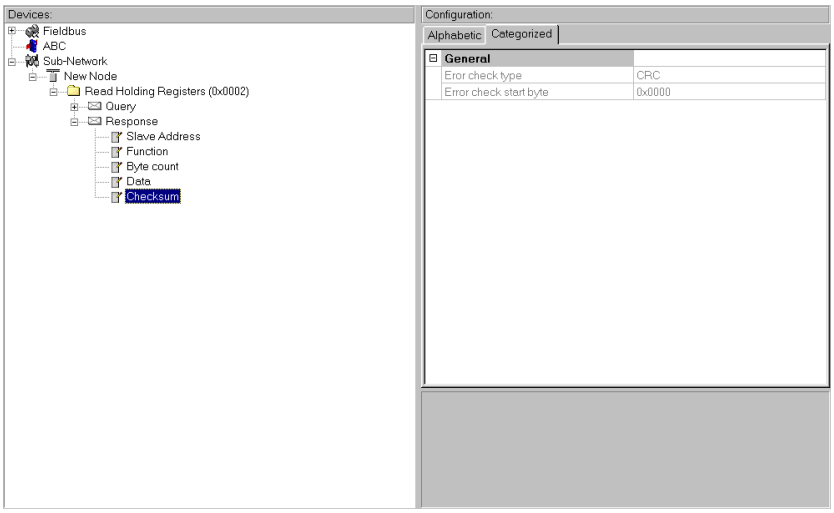
Column 3 in the Command Editor is where objects can be added for custom commands. Supported objects are Byte, Word, DWord, Data and Error Check. In this Modbus example it makes no sense to add an Error Check object since it is already incorporated in the standard frame but all other objects could be added in any way.

Response (E)

The "response" is defined much in the same way as the "query", with the difference that a "response" can depend on what is entered in the "query".

Query	1	2	3	4
	This column specifies the SlaveAddress.	This column specifies the Function Code in the Modbus message	(See below)	This column specifies the Error Check field.
DisplayName	Slave Address	Function	Data	Checksum
	(See Query)	(See Query)	(See Query)	(See Query)
Object Type	Byte	Byte	Data	Checksum
	(See Query)	(See Query)	(See Query)	(See Query)
Value	[SlaveAddress]	ID	User	Depend
	(See Query)	(See Query)	(See Query)	This object will get the same setting as the corresponding object in the Query. Further more, the object will appear as non-edit-able in the parameter window. (See below)

If 'Depend' is selected then this object in the "response" will get the same setting as the corresponding object in the "query", furthermore the object will appear as non-editable in the parameter window. (See below.)



Main Window

Sub Network Monitor

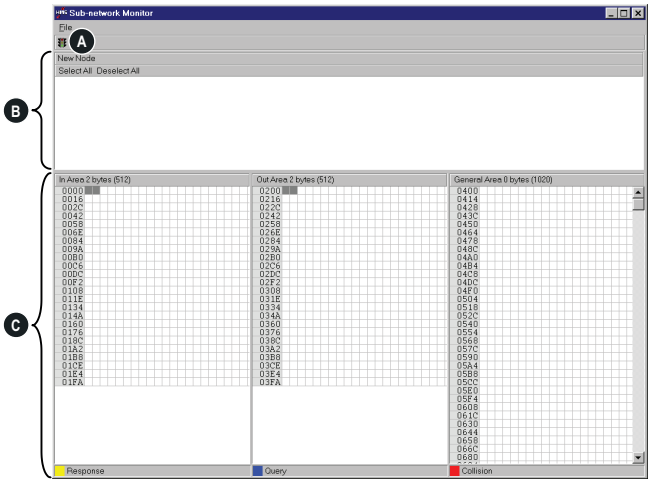
General

The Sub Network Monitor is intended to simplify configuration and troubleshooting of the sub network. It's main function is to display the data allocated for sub-network communication and detect if any area has been allocated twice, i.e if a collision has occurred.

All configured nodes, together with the commands, are listed in the middle of the screen (B). Selecting and deselecting commands makes it possible to view any combination of allocated data.

Note: The sub-network monitor has a negative influence on the overall performance of the ABC. Therefore the monitor functionality should be used with care.

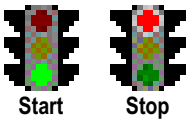
Operation



Sub-network monitor

A: Start / Stop sub network scanning

These icons are used to start / stop the scanning of the sub network. To stop the scanning, click on the red light. To start scanning again, simply click on the green light.



B: Nodes / Transactions

To view data blocks linked to a single command, select the command and the data will appear in the monitor area, see below. (C)

C: Monitor Area: Input / Output / General Data Areas

These areas display the data allocated in the Input, Output and General data areas. This information is colour coded as follows:

- **White** - No data allocated
- **Yellow** - Data allocated by a Response / Consume transaction
- **Blue** - Data allocated by a Query / Produce transaction
- **Red** - Collision. This area has been allocated more than once.
- **Grey** - Data allocated by the Control / Status registers

Node Monitor

General

The Node Monitor functionality provides an aid when setting up the communication with the slave-devices on the sub-network.

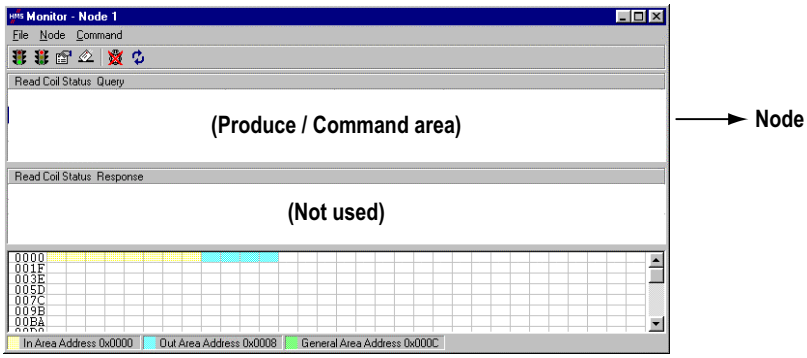
It offers an easy way of testing a specific command on a node, and monitor the result. It also provides an overview of the memory used by the node.

Note: Using the node monitor has a negative influence on the overall performance of the ABC. Therefore the monitor functionality should be used with care.

The Node Monitor behaves a bit different depending on which mode the ABC is currently running in.

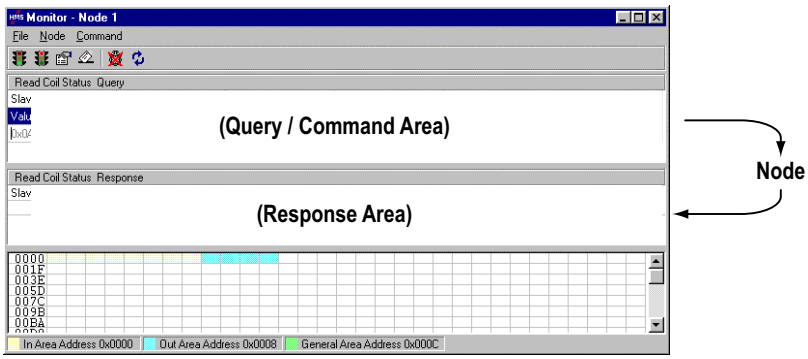
Generic Data Mode

In Generic Data Mode, the selected command is sent to the specified node. It is however not possible to monitor any response generated by the slave.

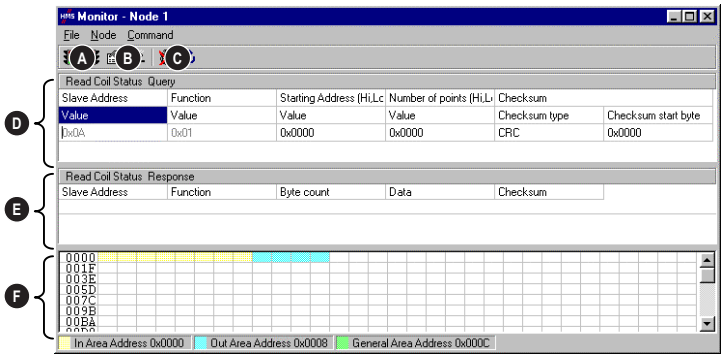


Master Mode

In Master Mode, the selected command is sent to the specified node. The result (Response) can be monitored in the Response area.

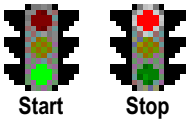


Operation



Node Monitor

A: Start / Stop Node Communication

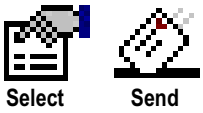


These icons are used to start / stop a node. Stopping is done by clicking the red light and could be seen as a temporary removal of the node, i.e no data will be sent to the node but it is still available. To start the node again, simply click on the green light.

This is a powerful feature when there is a problem with a particular node; the other nodes can be disconnected, helping to isolate the problem.

Note: If the Control and Status registers are enabled, the sub-net cannot be started or stopped without being activated from the fieldbus.

B: Select / Send Command



Select the command to monitor using the 'Select' icon, and click 'Send' to send the command.

C: Data Update ON / OFF

These icons are used to turn the monitor functionality ON or OFF. (See 'Monitor Area' below)

D: Command Area

This area displays the currently selected Command

E: Response Area (Master Mode only)

This area displays the response of a previously sent Command

F: Monitor Area

This area provides an overview of the data sent / received from the node. Areas in dark grey are reserved for the Status / Control registers.

Areas displayed in light grey are data objects used by the node. If data updating is enabled (See C: above) the contents of these areas are also displayed in hex.

Advanced Functions

Control and Status Registers

The Control / Status registers forms an interface for exchanging information between the fieldbus control system and the ABC.

The main purpose of these registers is to report sub-network related problems to the fieldbus control system. This interface is also used to ensure that only valid data is going out on the sub-network and that valid data is reported back to the fieldbus control system. See 20-4 “I/O-data during startup”.

Using these registers, it is also possible for the fieldbus control system to instruct the ABC to enable / disable specified nodes.

By default, these registers are located in the internal memory buffer at 0x000 - 0x001 (Status Register) and 0x200 - 0x201 (Control Register), however they can be disabled using the ABC Config Tool, see 13-5 “Status / Control bytes”. Disabling these registers will release the 2 reserved bytes in the internal memory buffer, however, the Status and Control functionality will not be available.

The handshaking procedure described on page 20-3 must be followed for all changes to these registers

Control Register (Fieldbus Control System -> ABC)

Byte 0 (Offset 0x200)								Byte 1 (Offset 0x201)							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			Control Code					Data							

Bits	Name	Description
15	Handshake Confirmation Bit (CR_HS_CONFIRM)	When the fieldbus control system has read the new information from the Status Register, it should set this bit to the same value as bit 15 in the Status Register
14	Handshake Toggle Bit (CR_HS_SEND)	The fieldbus control system should toggle this bit when new information has been written in the Control Register.
13	Data Valid (CR_DV)	This bit is used to indicate to the ABC if the data in the output data area is valid or not. The bit shall be set by the fieldbus control system when new data has been written. 1: Data Valid 0: Data NOT Valid
12 - 8	Control Code (CR_EC)	See table below.
7 - 0	Data (CR_ED)	See table below.

Control Codes

The following Control Codes are recognized by the ABC and can be used by the fieldbus control system.

Code	Name	Data	Description
0x10	DISABLE_NODE	Slave address of the node to disable	This instructs the ABC to disable a specific node from the sub network communication
0x11	ENABLE_NODE	Slave address of the node number to enable	This instructs the ABC to enable a specific node to be active in the sub network communication
0x12	ENABLE_NODES	Number of nodes to enable	This instructs the ABC to enable a number of nodes from a complete configuration

Status Register (ABC -> Fieldbus Control System)

Byte 0 (Offset 0x000)								Byte 1 (Offset 0x001)							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			Status Code					Data							

Bits	Name	Description
15	Handshake Toggle Bit (SR_HS_SEND)	The ABC toggles this bit when new information is available in the Status Register.
14	Handshake Confirmation Bit (SR_HS_CONFIRM)	When the ABC has read the new information from the Control Register, it sets this bit to the same value as bit 14 in the Control Register
13	Data Valid (SR_DV)	This bit Indicates to the fieldbus control system if the data in the input data area is valid or not. The bit is set by the ABC when new data has been written. 1: Data Valid 0: Data NOT Valid
12 - 8	Status Code (SR_EC)	Status code, see table below.
7 - 0	Data (SR_ED)	Status user data, see table below.

The Status Codes below are handled by the ABC and reported to the fieldbus control system using the Status Code and Data bits in the Status register. The meaning of these bits are different depending on the used communication model, see below.

Status Codes in Generic Data Mode

(The table below is valid only in Generic Data Mode.)

Code	Error	Data	Description
0x00	Invalid message	Number of messages	Incoming messages don't match any Consume instances
0x01	Frame error	-	End character is enabled, but no end character is received before message delimiter timeout
0x02	Consume Timeout	Number of instances	Number of Consume instances that has timed out
0x03	Overrun	-	If a receive buffer overflows, or an incoming message hasn't been handled before a new message has arrived
0x04	Other error	-	Other serial error (parity error, frame error)
0x1F	No error	-	Normal Condition

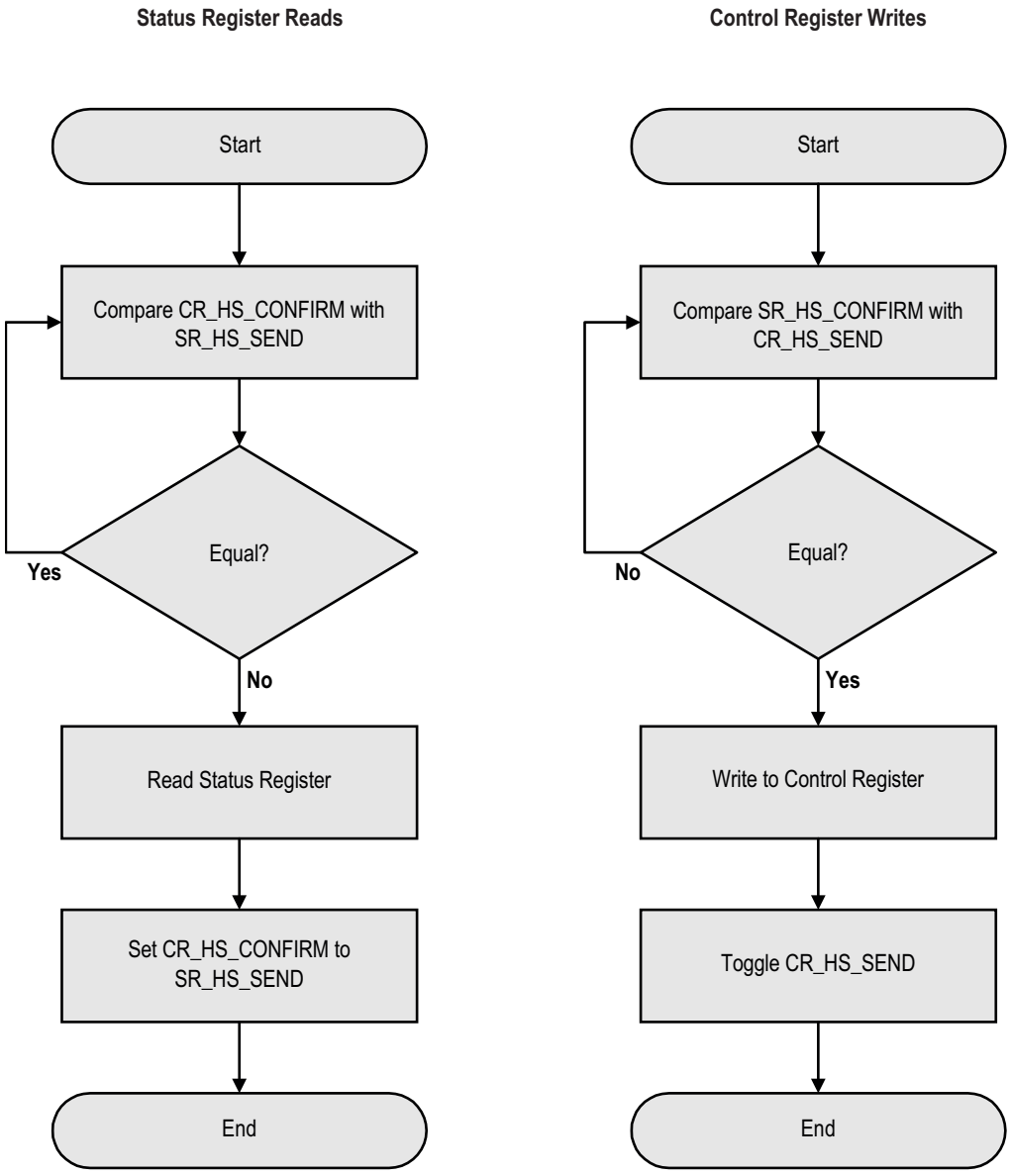
Status Codes in Master Mode

(The table below is valid only in Master Mode.)

Code	Error	Data	Description
0x00	Re-transmission	Number of re-transmissions	Reports the total number of re-transmissions on the subnetwork
0x01	Single node missing	Slave address of the missing node	Reports if a node is missing
0x02	Multiple nodes missing	Number of missing nodes	Reports if multiple nodes are missing
0x03	Overrun	Slave address of the node that sent too much data	Reports if more data than expected was received from a node
0x04	Other error	Slave address	Reports unidentified node
0x1F	No error	-	Normal Condition

Handshaking Procedure

The handshake bits are used to indicate any changes in the Status and Control Registers. The procedure below must be followed for all changes to these registers with the exception of the handshake bits themselves. (bits 14 and 15)

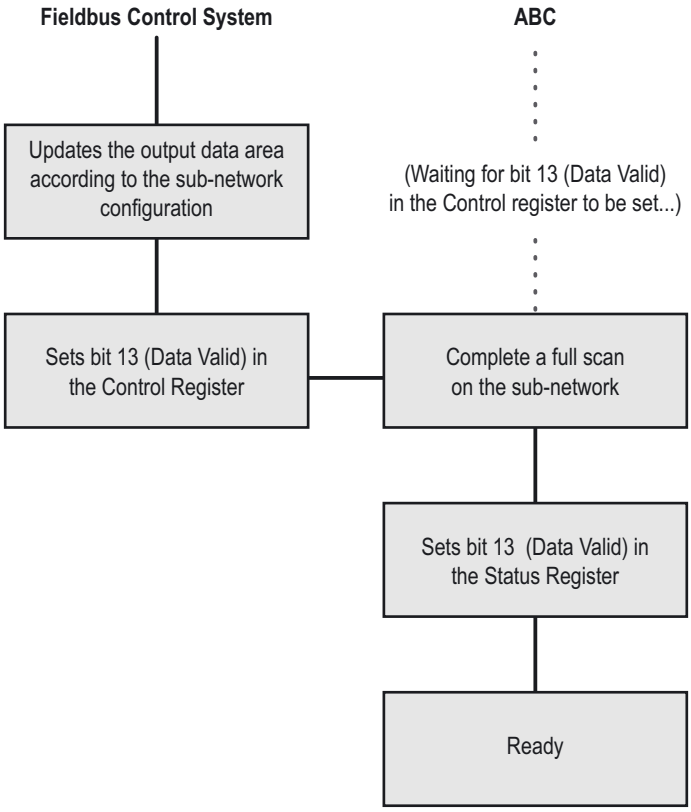


I/O-data during startup

Note: This section is only relevant when the Control / Handshake registers are enabled.

Bit 13 in the Control Register is used to ensure data consistency during start-up and at fieldbus off-line/ on-line transitions.

The bit should be treated as follows:



When the fieldbus changes from off-line to on-line state, the fieldbus control system should clear (0) the 'Data Valid' bit in the Control Register. The ABC will then clear the 'Data Valid' bit in the Status Register.

During startup, the ABC waits for the fieldbus control system to set the 'Data Valid' bit in the Control Register. Before this is done, it will not communicate with the devices on the sub network.

The 'Data Valid' bit in the Status Register may in some cases be delayed. This latency can be caused by a missing node or a bad connection to a node with a long timeout value assigned to it.

Therefore, the fieldbus control system should not wait for this bit to be set before communicating with the sub-network devices. It should be considered as an aid for the fieldbus control system to know when all data has been updated.

Note: As with all changes to these registers, the handshaking procedure (See "Handshaking Procedure" on page 3.) must be followed.

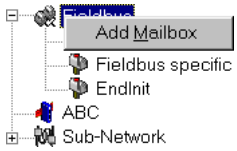
Advanced Fieldbus Configuration

The fieldbus interface of the ABC consists of an embedded AnyBus-S communication module. Normally, the AnyBus-S configuration settings are set up automatically by the ABC. However, advanced users can configure the AnyBus-S card for specific features. This chapter assumes that the reader is familiar with the AnyBus-S and it's application interface. For more information about the AnyBus-S platform, consult the AnyBus-S Design Guide.

The standard initialisation parameters are determined by the sub-network configuration. Information about the amount of input- and output data used for sub-network communication is used by ABC Config Tool to create the configuration message that sets the sizes of the input- and output data areas in the Dual Port RAM of the embedded AnyBus-S interface. It is possible to add fieldbus specific mailbox messages to customize the initialisation. This is done in the Mailbox Editor, see below.

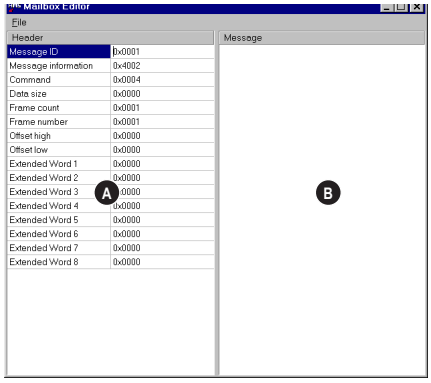
(A mailbox message is a HMS specific comand used for communication with an AnyBus-S module, consult the AnyBus-S Design Guide for more information.)

Mailbox Editor



The mailbox editor is accessed by right-clicking the fieldbus icon in the navigation window and then select Add Mailbox. Additional mailbox messages are be inserted between the 'StartInit' and 'EndInit' messages.

A mailbox message consists of a Header section and a data section where the Header consists of 16 words (32 bytes) and the data section consists of up to 128 words (256 bytes). All fields are editable except the Message information field that is fixed to 0x4002, which means that only fieldbus specific mailbox messages can be entered here.



Mailbox Editor

The mailbox message is presented as two columns; one contains header information (A), the other one contains the message data (B).

To add message data, simply change the Data size parameter in the header column (A), and the corresponding number of bytes will appear in the message data column (B).

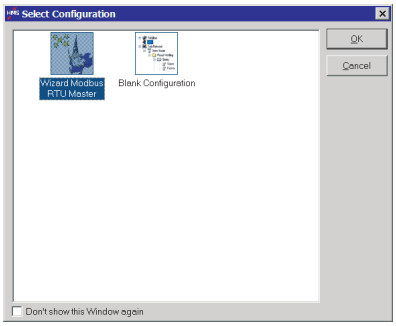
For more information about fieldbus specific mailbox messages, consult the separate AnyBus-S Fieldbus Appendix for the fieldbus you are using. For general information about the AnyBus-S platform, consult the AnyBus-S Design Guide.

Configuration Wizards

The wizard option automatically creates a sub network configuration based information entered by the user, i.e. the user simply has to "fill in the blanks". Note that this will only work when the sub network fits the wizard profile, in all other cases the 'Blank Configuration' option must be used.

The online help system explains each configuration step in detail.

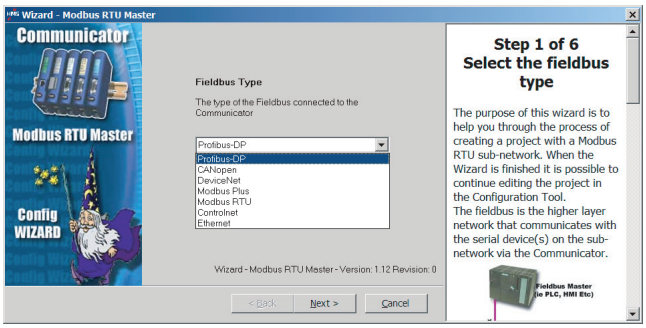
Select Wizard Profile



First, select a profile suitable for the sub network.

- **Wizard Modbus RTU Master**
  Suitable for Modbus slave devices.
- **Blank Configuration**
  Manually configure the sub network.

Step 1: Communicator Type

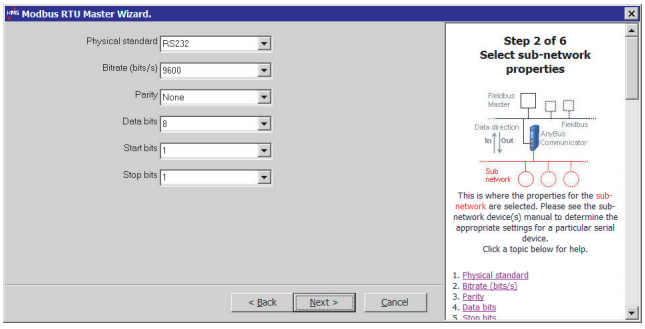


Select 'Profibus-DP'.

Click 'Next' to continue.

Tip: It is possible to return to a previous menu at any time without losing any settings by clicking 'Previous'.

Step 2: Physical Settings



Select the physical properties of the sub network.

Click 'Next' to continue.

Steps 3 - 6

Consult the on line help system for further information.

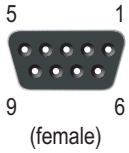
Troubleshooting

Problem	Solution
Problem during configuration Upload / Download. The Config Line “led” turns red in the ABC Config Tool.	• Serial communication failed. Try again
The serial port seems to be available, but it is not possible to connect to the ABC	• The serial port may be in use by another application. Exit the ABC Config Tool and close all other applications including the ones in the system tray. Try again • Select another serial port Try again
Poor performance	• Right click ‘Sub-Network’ in the Navigation window and select ‘Sub-Network Status’ to see status / diagnostic information about the sub network. If the ABC reports very many re-transmissions, check your cabling and / or try a lower baud rate setting for the sub network (if possible). • Is the Sub-Net Monitor in the ABC Config Tool active? The sub-network monitor has a negative influence on the overall performance of the ABC, and should only be used when necessary. • Is the Node Monitor in the ABC Config Tool active? The node monitor has a negative influence on the overall performance of the ABC, and should only be used when necessary.

Connector Pin Assignments

Profibus Connector

Pin	Signal	Description
Housing	Shield	Bus cable shield, connected to PE
1	-	-
2	-	-
3	B-Line	Positive RxD/TxD (RS485)
4	RTS[a]	Request To Send
5	GNDBUS[b]	Isolated GND from RS-485 side
6	+5V BUS[b]	Isolated +5V output from RS-485 side (80mA max)
7	-	-
8	A-Line	Negative RxD/TxD (RS485)
9	-	-

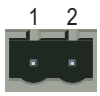


a. Used in some equipment to determine the direction of transmission. However, in normal applications only A-Line, B-Line and Shield are used.
b. Used for bus termination. Some devices such as optical transceivers (RS485 to fibre optics) may require power from these pins.

Recomended Profibus connectors:
Profibus Max standard, part nr 134928 and Pofibus reversed, part nr 104577 from www.erni.com
Fast connect Bus connector, part nr, 6GK1500-0FC00 or 6ES7 972-0BA50-0XA0 from www.siemens.com

Power connector

Pin	Description
1	+24V DC
2	GND



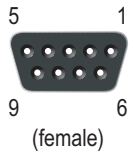
Note:

- Use 60/75 or 75×C copper (CU) wire only.
- The terminal tightening torque must be between 5-7 lbs-in (0,5-0,8 Nm)

Sub-network connector

This connector is a standard DSUB9 female and is used to connect the ABC to the sub-network. Based on the configuration selected in the ABC Config software, the corresponding signals are activated.

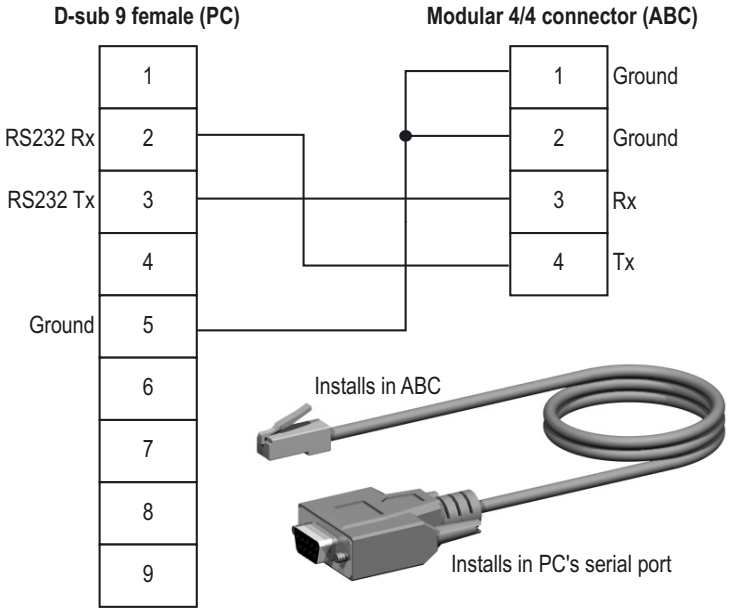
Pin	Description	RS232	RS422	RS485
1	+5V Output(50mA max)	✓	✓	✓
2	RS232 Rx	✓		
3	RS232 Tx	✓		
4	Not connected			
5	Ground	✓	✓	✓
6	RS422 Rx +		✓	
7	RS422 Rx -		✓	
8	RS485 + /RS422 Tx+		✓	✓
9	RS485 - /RS422 Tx-		✓	✓



PC connector

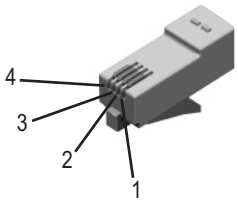
Configuration Cable Wiring

A cable can be purchased from HMS Industrial Networks (It is included in part. no 017620).



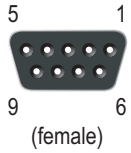
Modular 4/4 (ABC)

Pin	Description
1	Signal ground
2	Signal ground
3	RS232 Rx, data input to ABC
4	RS232 Tx, data output from ABC



DSUB 9 (PC)

Pin	Description
1	Not connected
2	RS232 Rx, data input to PC
3	RS232 Tx, data output from PC
4	Not connected
5	Ground
6 - 9	Not connected



Technical Specification

Mechanical

Housing

Plastic housing with snap-on connection to DIN-rail, protection class IP20

Dimensions

120 mm x 75 mm x 27 mm, L x W x H (inches: 4,72" x 2,95" x 1,06"; L x W x H)

Electrical Characteristics

Power Supply

Power: 24V ± 10%

Power Consumption

Maximum power consumption is 280 mA on 24V. Typically around 100 mA

Environmental

Relative Humidity

The product is designed for a relative humidity of 0 to 95% non-condensing

Temperature

Operating:	-5°C to +55°C
Non Operating:	-55°C to +85°C

EMC Compliance

CE-mark

Certified according to European standards unless otherwise is stated

Emission	Immunity
According to EN 50081-2:1993	According to EN 61000-6-2:1999

UL/c-UL compliance

This unit is an open type listed by the Underwriters Laboratories. The certification has been documented by UL in file E214107.

ASCII Table

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	NUL 0	SOH 1	STX 2	ETX 3	EOT 4	ENQ 5	ACK 6	BEL 7	BS 8	HT 9	LF 10	VT 11	FF 12	CR 13	SO 14	SI 15
1x	DLE 16	DC1 17	DC2 18	DC3 19	DC4 20	NAK 21	SYN 22	ETB 23	CAN 24	EM 25	SUB 26	ESC 27	FS 28	GS 29	RS 30	US 31
2x	(sp) 32	! 33	" 34	# 35	$ 36	% 37	& 38	' 39	(40	) 41	* 42	+ 43	, 44	- 45	. 46	/ 47
3x	0 48	1 49	2 50	3 51	4 52	5 53	6 54	7 55	8 56	9 57	: 58	; 59	< 60	= 61	> 62	? 63
4x	@ 64	A 65	B 66	C 67	D 68	E 69	F 70	G 71	H 72	I 73	J 74	K 75	L 76	M 77	N 78	O 79
5x	P 80	Q 81	R 82	S 83	T 84	U 85	V 86	W 87	X 88	Y 89	Z 90	[91	\ 92	] 93	^ 94	_ 95
6x	` 96	a 97	b 98	c 99	d 100	e 101	f 102	g 103	h 104	i 105	j 106	k 107	l 108	m 109	n 110	o 111
7x	p 112	q 113	r 114	s 115	t 116	u 117	v 118	w 119	x 120	y 121	z 122	{ 123	| 124	} 125	~ 126	DEL 127